

Modèles pour la simulation

Cours 1:

Modélisation Multi- Niveaux et Introduction en SystemC

Daniela Genius

Master SESI 2ème année
Sorbonne Université
France

Modèles pour la simulation

Daniela Genius Radermacher

Niveau d'abstraction		Simulation	Vérification formelle	Optimisation
Système	Fonctions + Canaux	SYSML / AADL		
		KPN / SDF		SDF -> prog linéaire - calcul debit SDF - optim buffer - GLPK
	+ temps			
	Etiquettes temps			
	Intro horloge			
	Codage binaire			
	Cycle horloge			

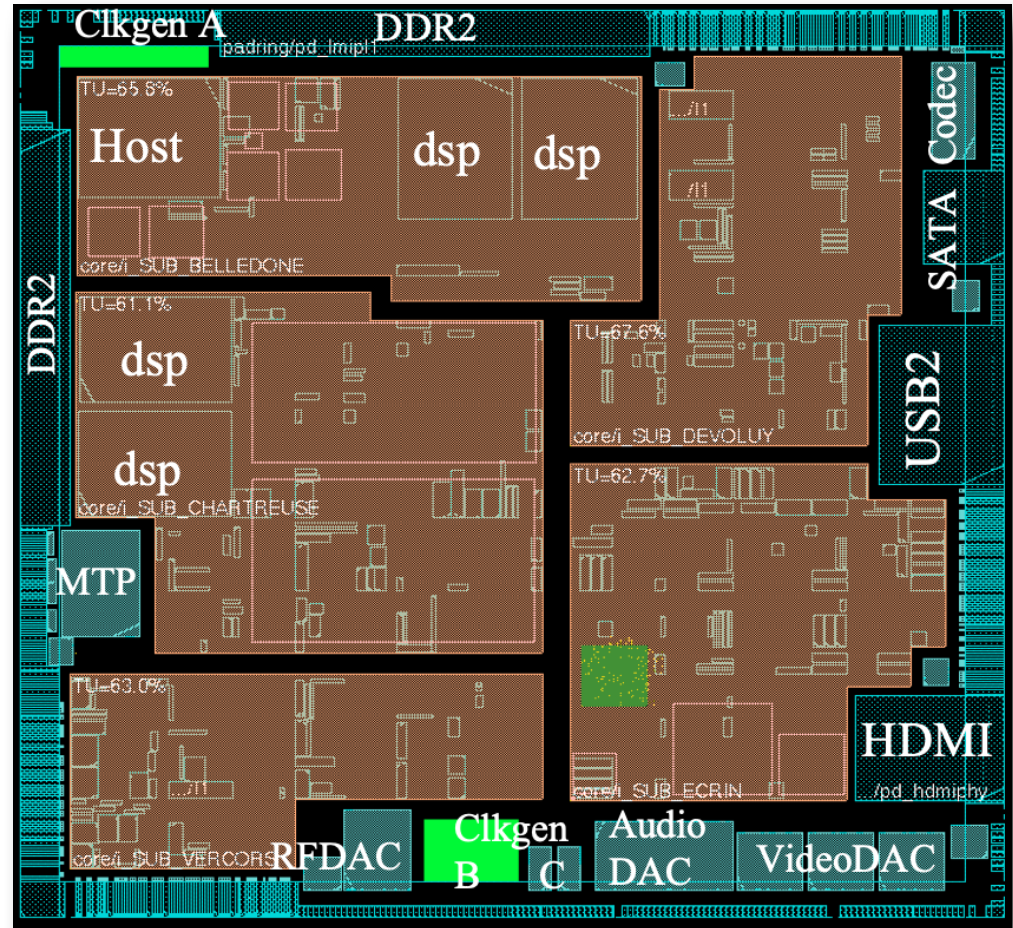
Mécanismes de simulation	TLM	Comportements du système = Automate
	CABA	- traces d'exécution
	RTL	- propriétés
		- évaluation prp - TP VIS et UVM

Contenu

- Introduction (Cours 1)
 - Modélisation multi-niveaux
 - Introduction en SystemC
- Mini Risc-V au niveau RTL (Cours 2)
- Le niveau CABA (Cours 3)
- SystemC-AMS (Cours4)

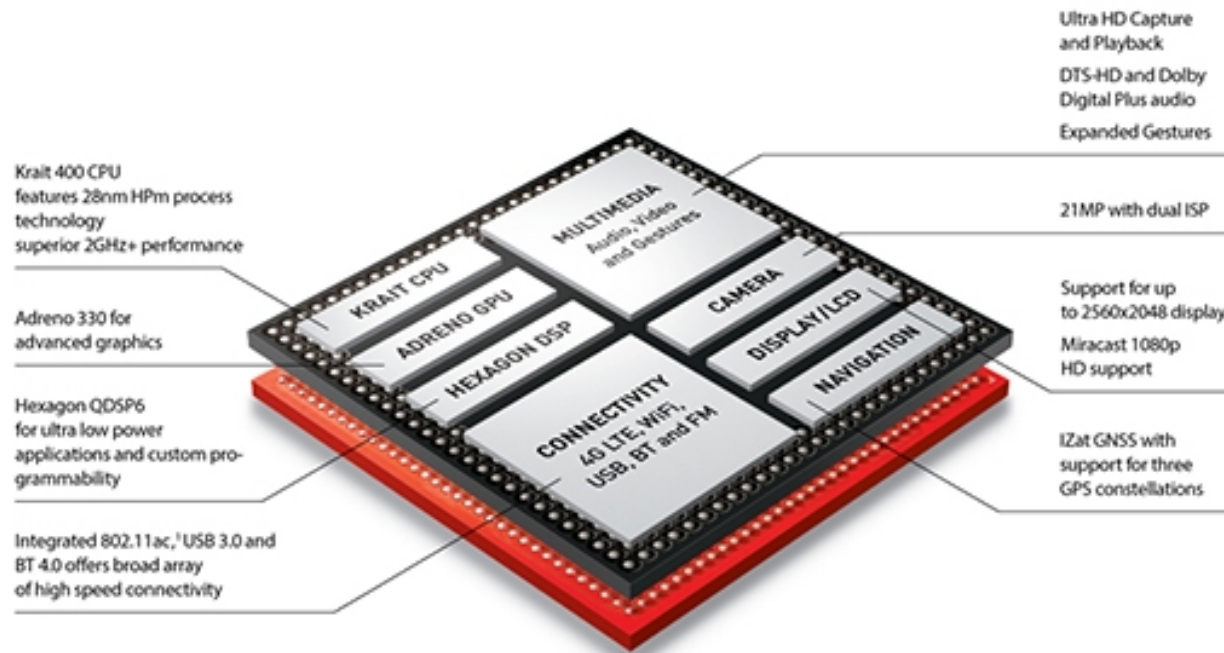
Complexité : Exemple Décodeur “HD 264”

- ② Circuit STMicro dédié au décodage de la TV HD (norme H264)
 - Circuit contenant 150M transistors et 886 pads IOs (~5 GMIPS)
 - 128 sources d'interruption
 - 73 initiateurs et 96 cibles
 - 115 réseaux d'horloge (19 pour les interconnexions)
- ② Construction du système
 - 4 processeurs (2 DSP vidéo, 1 DSP audio 1 uP généraliste pour configuration du système)
 - 36 Soft IPs + 2 Hard IPs
 - 140 memory cuts



Circuit développé en 200X ?!
Que fait on alors aujourd'hui ...

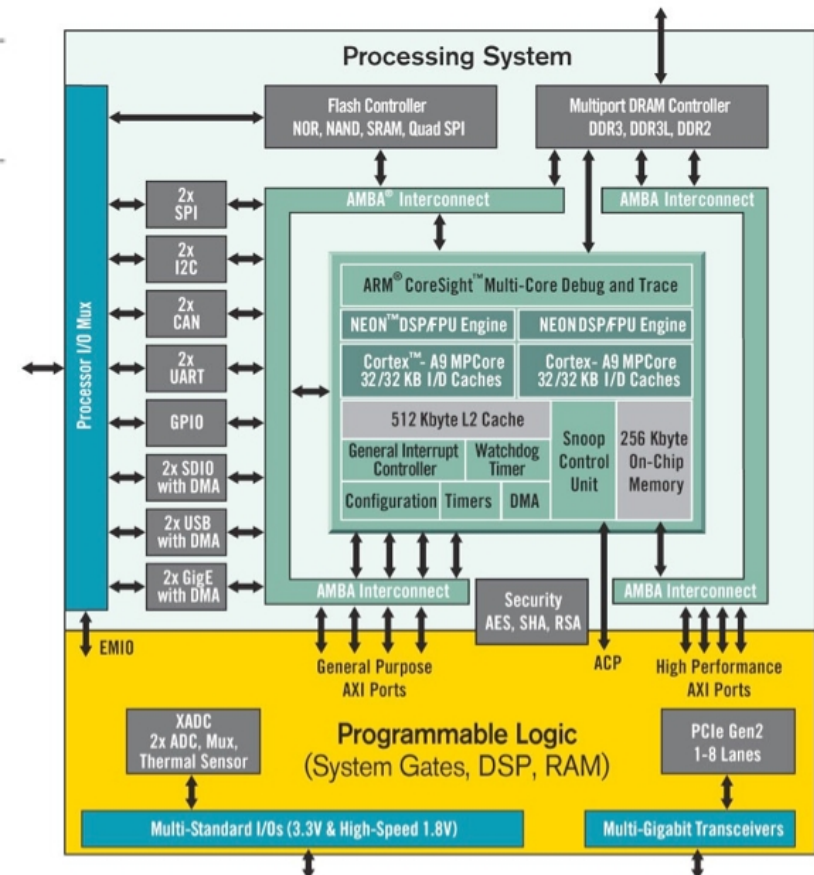
Les SoCs numériques, une architecture conjointe



La majorité des SoCs intègrent des blocs numériques dédiés + des processeurs

Il en va de même pour les circuits FPGA (Xilinx & Altera)

Regrouper au sein d'une même puce => performances & fiabilité

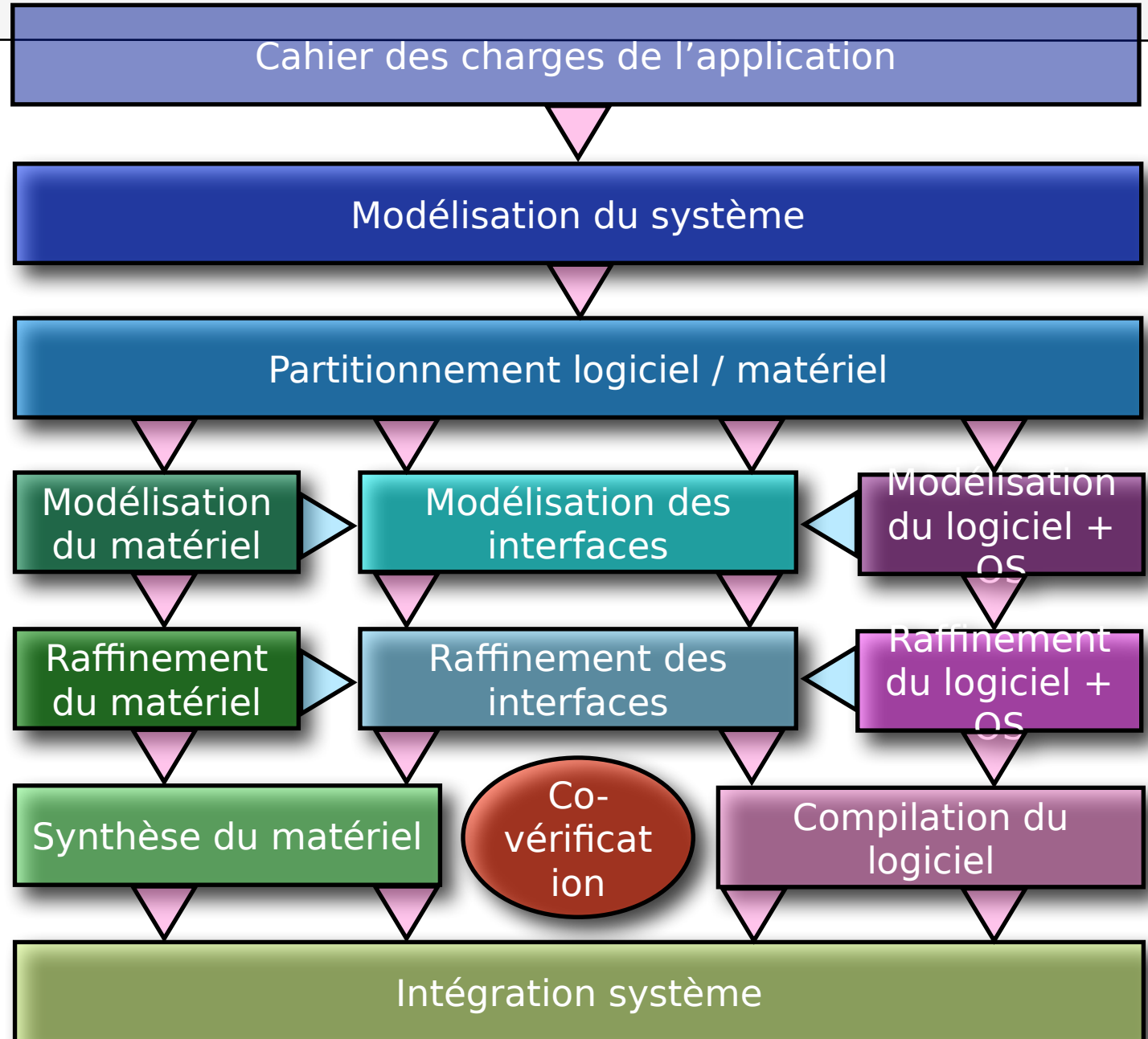
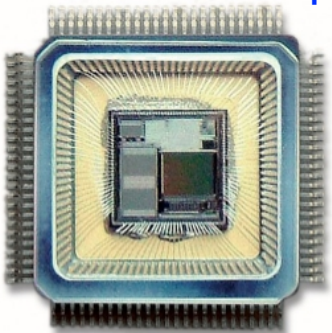


Flot de développement de niveau système

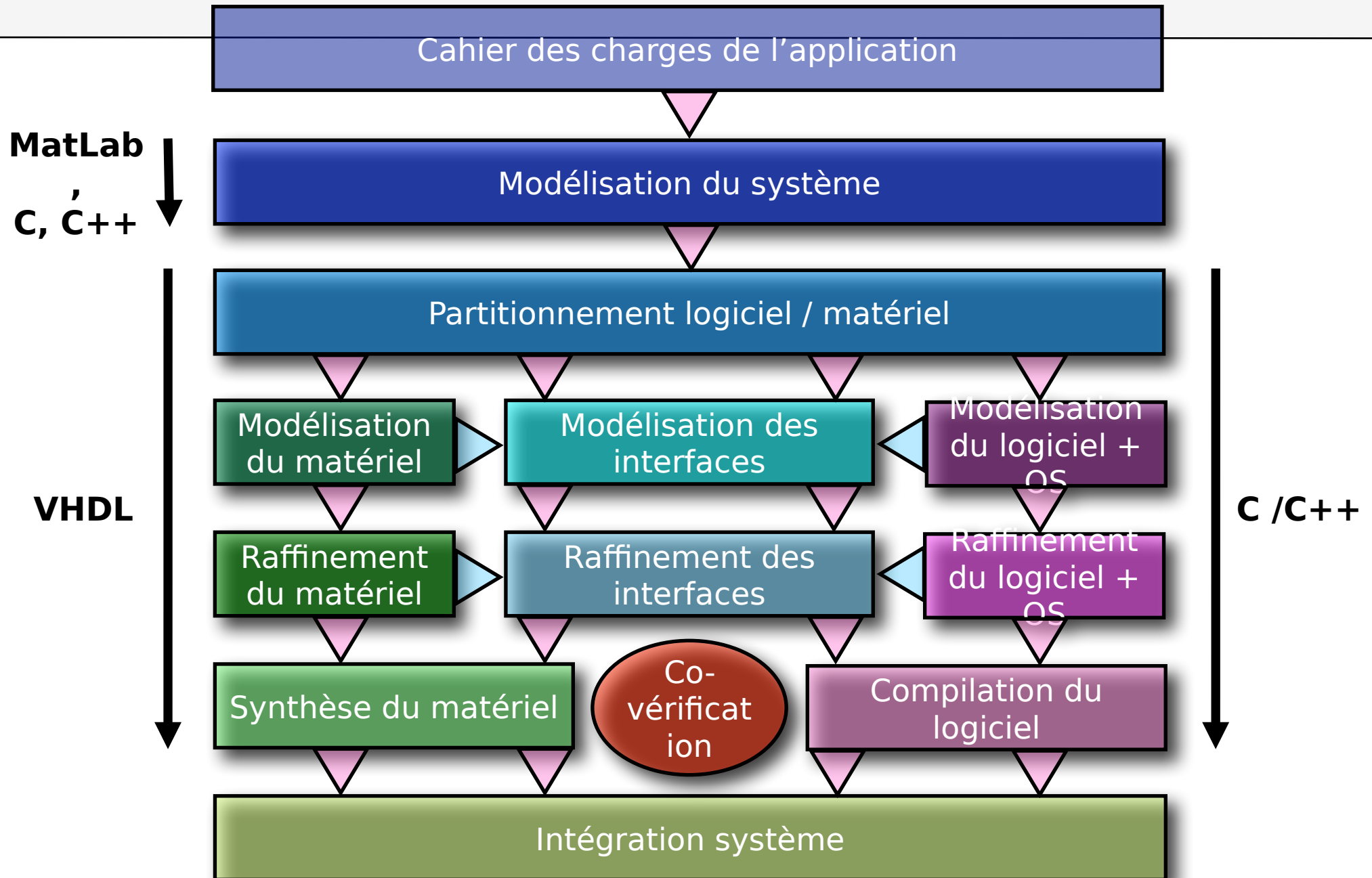
Spécifications



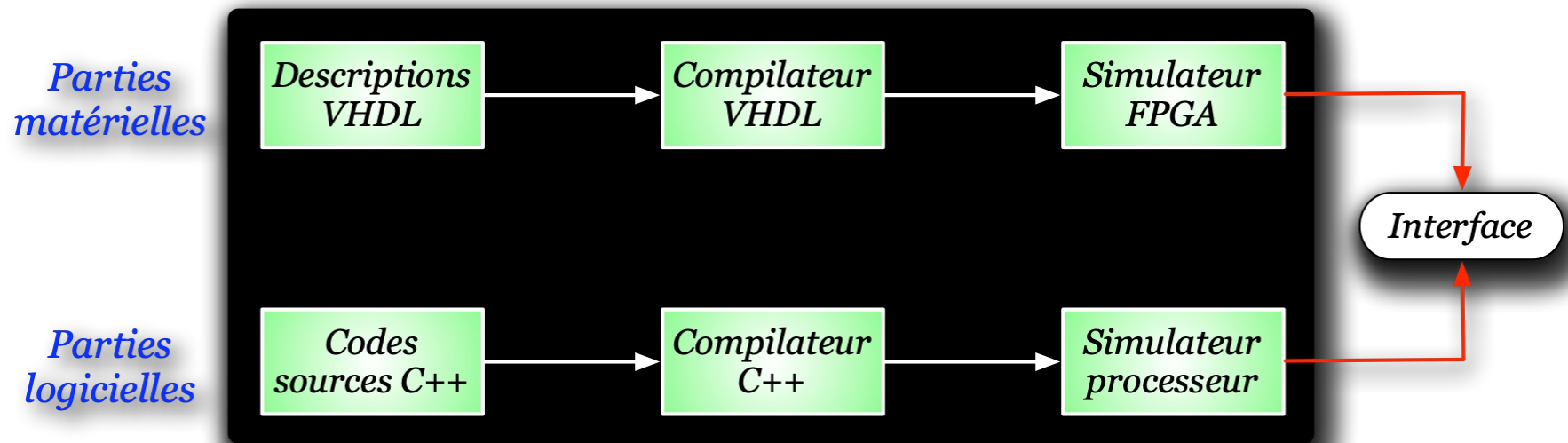
Circuit numérique



Flot de conception système usuel



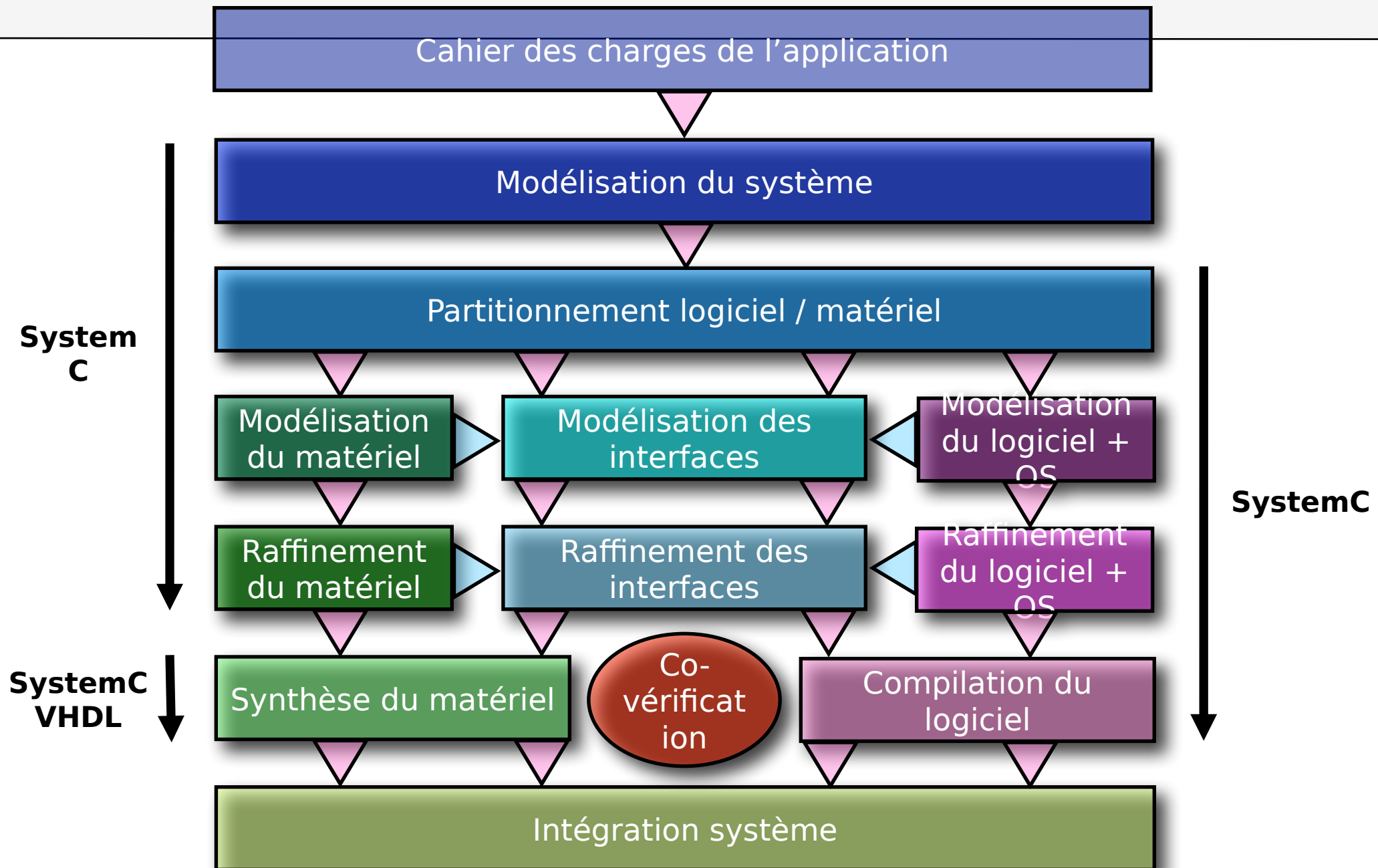
Simulation lors de conceptions conjointes (usuelle)



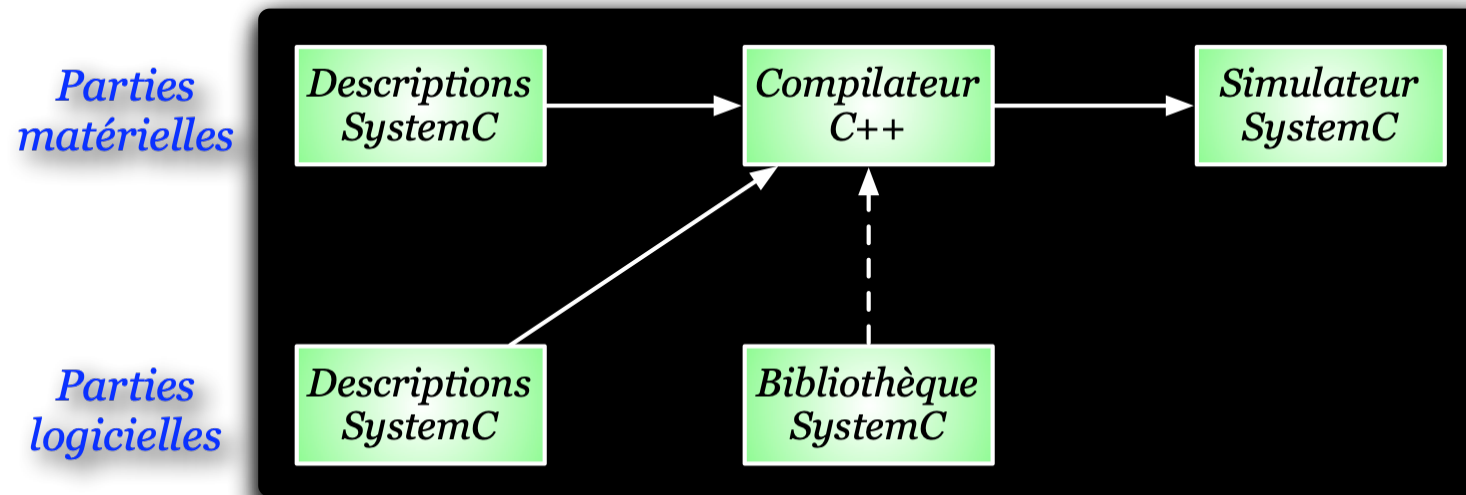
Lors de la conception d'un système hétérogène, il est nécessaire de disposer de pléthore d'outils différents peu faits pour communiquer ensemble => Problème majeur.

De plus les interfaces de communication (entre outils) ralentissent grandement des simulations déjà longues !

Evolution du flot de conception à l'aide de SystemC



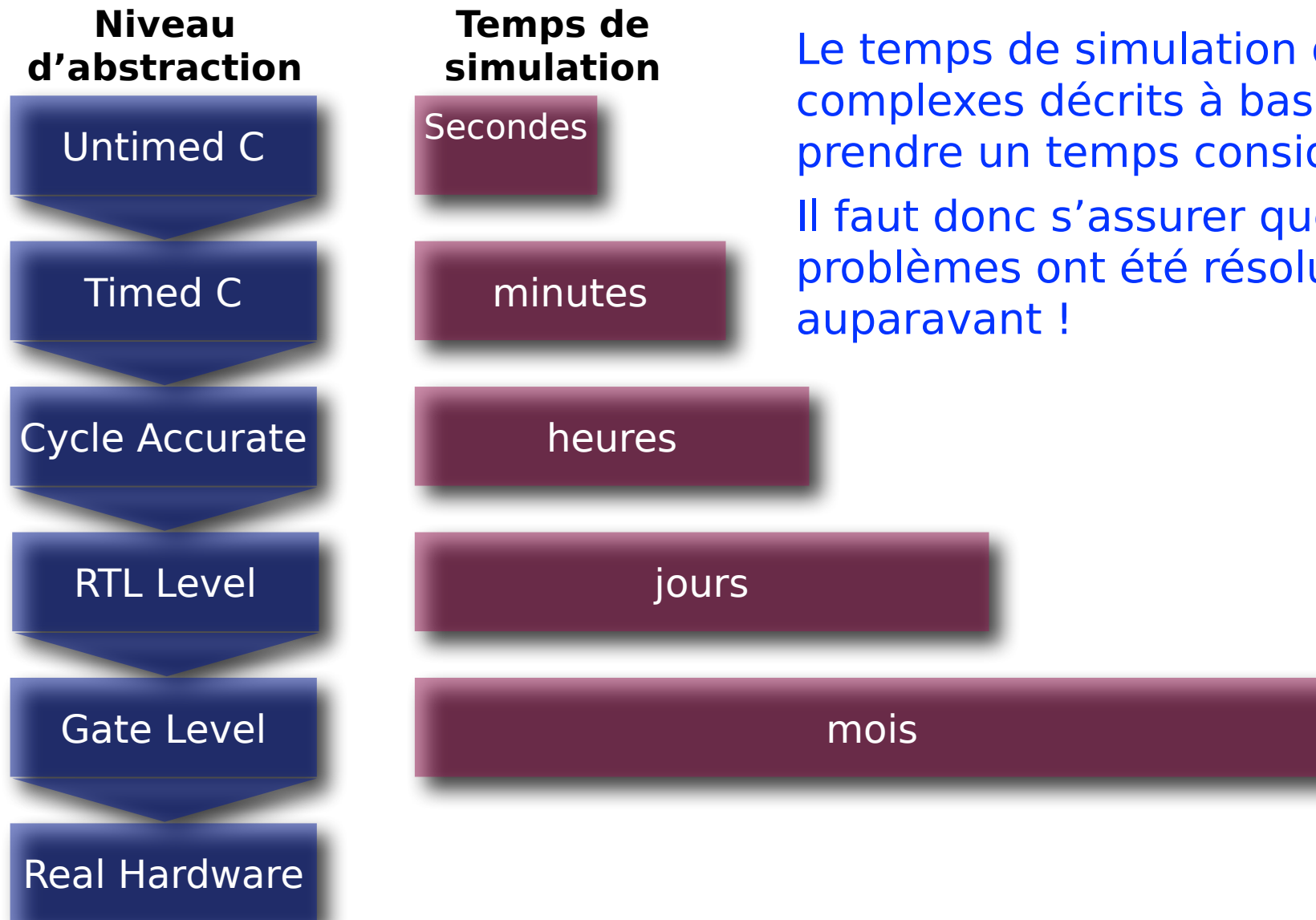
Simulation lors de conceptions conjointes (SystemC)



Le langage SystemC permet de simplifier les étapes de conception conjointe en unifiant les outils de conception.

En même temps il permet de conserver un langage unique pour développer du logiciel et du matériel (moins de changements de langages implique moins d'erreurs de "traduction")

Evolution du temps de simulation des modèles



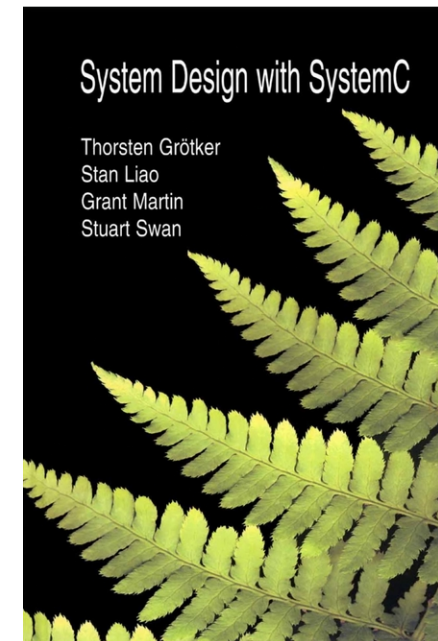
Le temps de simulation de systèmes complexes décrits à bas niveau peut prendre un temps considérable...

Il faut donc s'assurer que les problèmes ont été résolus auparavant !

Aspects de précision (anglais : accuracy)

- Structural Accuracy/Précision structurelle
- Timing accuracy/Précision temporelle
- Functional accuracy/Précision temporelle
- Data organization accuracy/Précision de l'organisation des données
- Communication protocol accuracy

- *Source : T. Grötke, System Design*
- *with SystemC 2002, Springer US*



Structural Accuracy/Précision Structurelle

- = Dans quel mesure le modèle reflète-t-il la vraie structure de l'implémentation ?
- Partitionnement matériel-logiciel : les modules matériels et logiciels peuvent être identifiés comme tels et distingués ?
- Les signaux et pins sont-ils représentés dans le modèle ?
- Communication entre tâches, est-elle donnée au niveau de précision des mécanismes du système d'exploitation temps-réel ?

Timing Accuracy/ Précision Temporelle

- = Mesure dans laquelle le modèle reflète le timing de l'application réelle
- Valeurs précis ou approximatives en temps absolu ?
- Temps réel (nanosecondes) ou cycles d'horloge ?
-
- Remarque : précision au niveau cycle d'horloge est le plus souvent suffisant car les outils d'implémentation s'occupent des détails

Functional Accuracy/Précision Fonctionnelle

= Mesure dans laquelle le modèle reflète la fonctionnalité de l'application réelle

- Omettre de la fonctionnalité complexe afin de simplifier le modèle et/ou augmenter la vitesse de simulation
- Exemple 1 : modèle de bus simplifié pour communication
- Exemple 2 : utilisation de l'arithmétique à virgule flottant au lieu de l'arithmétique à virgule fixe

- = Mesure dans laquelle la modèle reflète l'organisation réelle de données dans l'implémentation
- Des structures de donnée sont-elle représentées dans le modèle du tout/dans quelle mesure ?
- La disposition spatiale de données est elle représentée dans le modèle du tout/dans quelle mesure ?

Communication Protocol Accuracy/Précision du protocole de communication

- = Mesure dans laquelle le protocole de communication de l'application réelle utilisé dans l'implémentation cible est représenté
- Exemples : Détails du protocole FIFO au niveau cycle précis bit précis
 - ports, cycles d'horloge
- Communications au niveau transactionnel (FIFO, bus)
- Niveau transactionnel (TLM) : modules comme des classes interfaces; transactions s'effectuent par des appels à des fonctions interfaces

Quel niveau d'abstraction choisir ?

- Tous les modules d'un système ne sont pas forcément modélisés au même niveau d'abstraction
- Un module en phase de test par exemple peut être représenté à un niveau très détaillé, tandis que le test-bench utilise des modèles très abstraits
- Inversement, partant d'un modèle au niveau d'implémentation très détaillé (exemple VHDL), le concepteur peut avoir intérêt à créer un modèle plus abstrait pour la simulation rapide, et pour la protection de la propriété intellectuelle
 - projet SoCLib (2010's): interdiction d'avoir des modèles au niveau RTL
- Le concepteur peut être amené à raffiner successivement l'un des modules d'une spécification haut niveau ou purement fonctionnelle jusqu'à un niveau RTL, tandis que d'autres modules restent à haut niveau d'abstraction

Les niveaux d'abstraction et leurs spécificités

Modèle algorithmique

Modèle UnTimed Functional
(UTF)

Modèle Timed Functional (TF)

Modèle Bus Cycled Accurate
(BCA)

Modèle Cycled Accurate (CA)

Niveau transfert de registres
(RTL)

Les niveaux d'abstraction et leurs spécificités

Modèle algorithmique

Modèle UnTimed Functional
(UTF)

Pas de notion de temps et
pas de notion de transfert
des données.

Modèle Timed Functional (TF)

Modèle Bus Cycled Accurate
(BCA)

Notion de temps dans les
processus et les transferts de
données (modélisation gros
grains)

Modèle Cycled Accurate (CA)

Niveau transfert de registres
(RTL)

Les processus et les
signaux sont au cycle
près et au bit près.

Les niveaux d'abstraction et leurs spécificités

Modèle algorithmique

Vérification fonctionnelle
Exploration algorithmique
Vérification algorithmique

Modèle UnTimed Functional
(UTF)

Modèle Timed Functional (TF)

Benchmarking des performances
Analyse des architectures
Développement des parties logicielles

Modèle Bus Cycled Accurate
(BCA)

Modèle Cycled Accurate (CA)

Estimation précise des performances
Développement des drivers
Développement des micro-architectures

Niveau transfert de registres
(RTL)

Les niveaux d'abstraction et leurs spécificités

Modèle algorithmique

- Système spécifié à haut niveau d'abstraction
- Indépendant de l'implémentation

Les niveaux d'abstraction et leurs spécificités

Modèle UnTimed Functional (UTF)

- Aucune notion de temps/de délai
- Tous les communications sont point à point
- Les liens de communications (bus) sont modélisés

Les niveaux d'abstraction et leurs spécificités

Modèle Timed Functional (TF)

- Les communications sont toujours point à point
- Des délais sont ajoutés aux processus afin de représenter des contraintes temporelles d'une implémentation cible spécifique
- Rend possible l'analyse précoce de la partition matérielle-logicielle (trade-off)

Les niveaux d'abstraction et leurs spécificités

- Précis aux frontières des modules (boundaries)
- Pas précis au cycle
- Ne reflète pas la structure de l'implémentation cible
- Communication entre modules par appels de fonctions, pas précis au niveau structure

Modèle Bus Cycled Accurate
(BCA)

Les niveaux d'abstraction et leurs spécificités

- Pin et cycle précis aux frontières
- Structure interne ne reflète pas nécessairement l'implémentation cible

Modèle Cycled Accurate (CA)

Les niveaux d'abstraction et leurs spécificités

- Communication entre processus par signaux
- Registres et logique combinatoire fidèlement représentés
- Pin et cycle précis
- Pas de gate delays
- Exemple: Verilog, VHDL

Niveau transfert de registres
(RTL)

Models of Computation (MoC)

- **Questions à se poser :**

- Comment le temps est-il modélisé (untimed, integer, real) ?
- Comment les évènements sont-ils ordonnés (ordre partiel, ordre global) ?
- Quelles méthodes de communication entre processus concurrents sont supportées ?
- Quelles sont les règles d'activation pour les processus ?
- Exemples : réseaux de Petri, Kahn Process Networks (KPN), Synchronous Data Flow (SDF)

- **Deux constats :**

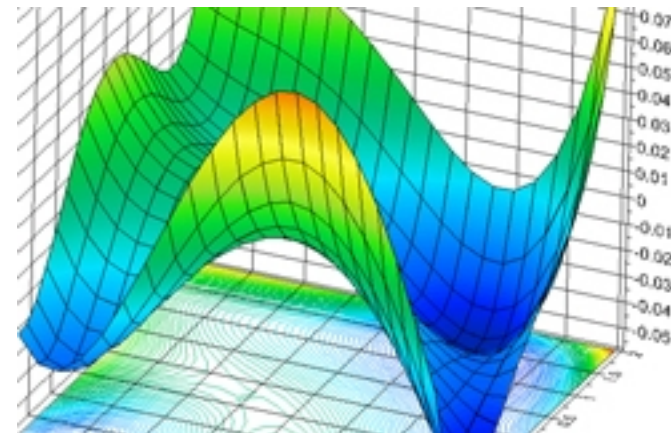
- *Tandis que VHDL et Verilog n'ont qu'un seul MoC, en SystemC, des MoC peuvent être ajouté dans une couche supérieure.*
- Le MoC de base de SystemC est très généraliste
- (à la fin du cours, nous allons étudier d'autres MoC : SystemC-AMS pour simulation analogique/mixte)

Conclusion

- ⊙ L'étude de performance se fait à l'aide d'un seul modèle,
 - Implantation logicielle,
 - Implantation matérielle,
- ⊙ Un seul modèle, et un seul langage permettent,
 - Gain en temps de développement,
 - Une cohérence dans l'évaluation (toujours les mêmes tests, les mêmes résultats)
- ⊙ Exploration «rapide» de diverses solutions.



Evaluation de diverses solutions



Introduction en SystemC



Merci à Bertrand Le Gal (IMS et Université de Bordeaux) pour la reprise d'une partie de son cours

Pourquoi utiliser SystemC ?

- ⊙ SystemC est un HDL (*Hardware Description Language*) qui a pour objectif de modéliser des systèmes numériques:

 - Le langage permet la modélisation des **systèmes matériels**

 - Le langage permet la modélisation des **systèmes logiciels**

 - Le langage permet la modélisation des **systèmes mixtes** (logiciels + matériels)

- ⊙ L'objectif de la modélisation d'un système est :

 - De vérifier l'exactitude du comportement attendu

 - D'analyser et optimiser ses performances

 - D'étudier l'impact de certaines décisions (i.e. découpage logiciel/matériel)

 - De permettre le dimensionnement et optimiser l'implantation

Introduction à SystemC

- ⊙ SystemC n'est pas exactement un nouveau langage

 - Il s'agit d'une **bibliothèque objet décrite en langage C++**

 - Un modèle SystemC est donc un programme écrit en C++

 - ▶ Utilise les classes de la bibliothèque logicielle SystemC

- ⊙ C'est le fruit de contributions de plusieurs sociétés

 - Synopsys met son outil commercial Scenic dans le domaine libre en 1989 et crée la version 0.9 de SystemC

 - Frontier Design y apporte sa contribution qui donne lieu à la version 1.0

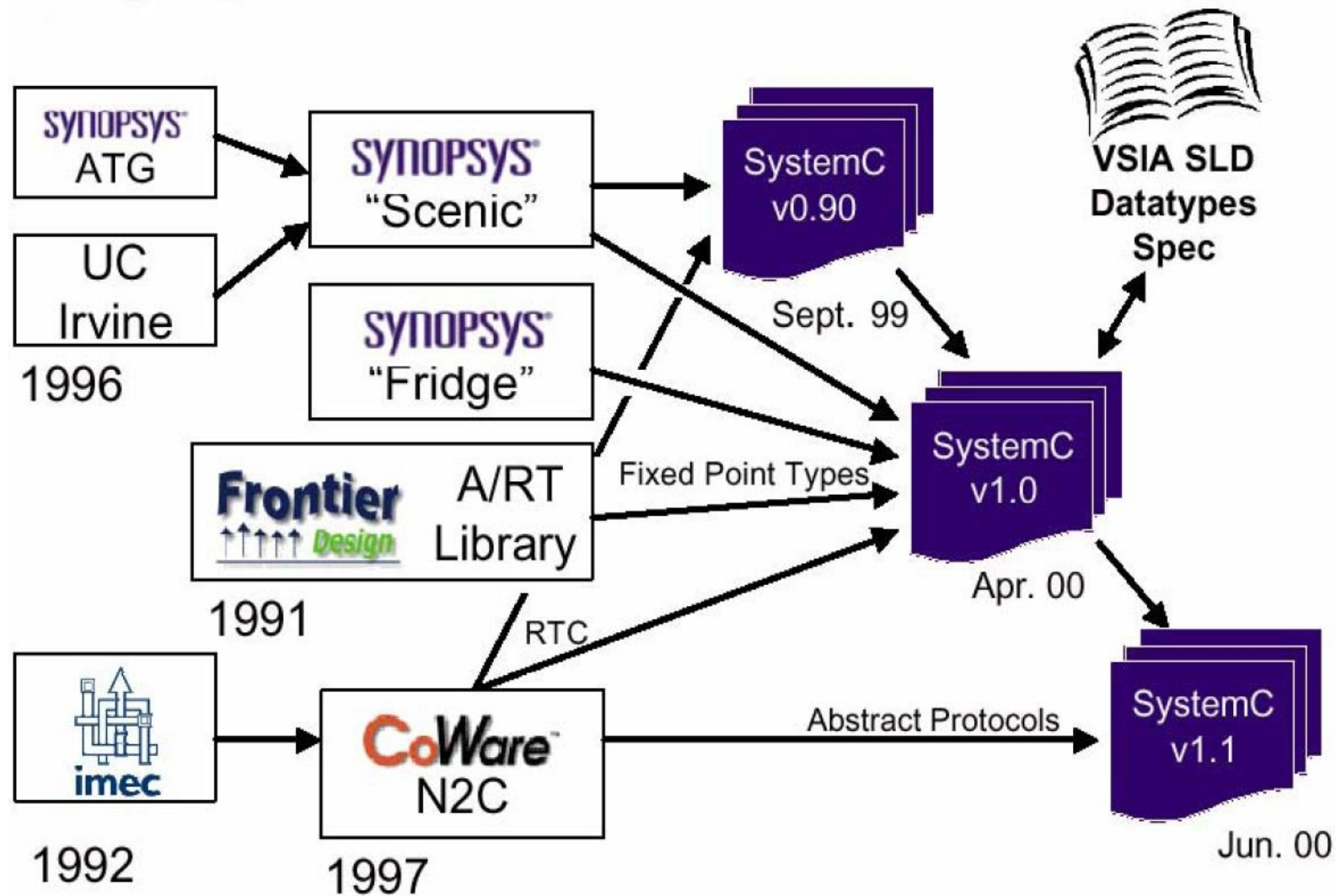
 - CoWare apporte sa contribution et aboutit en 2000 à la version 1.1

- ⊙ L'OSCI (*Open SystemC Initiative*)

 - Rassemble une multitude de sociétés et laboratoires de recherche.

 - En charge de diffuser et de rédiger les spécifications de SystemC.
Fusionné avec Accellera fin 2011

SystemC Héritage



Evolution

- ▲ **SystemC1.0**
 - ▲ Provide VHDL like capabilities
 - ▲ Simulation kernel
 - ▲ Fixed point arithmetic data types
 - ▲ Signals (communication channels)
 - ▲ Modules
 - ▲ Break down designs into smaller parts
- ▲ **SystemC2.0 (2003)**
 - ▲ Complete library rewrite to upgrade into true SLDL
 - ▲ Events as primitive behavior triggers
 - ▲ Channels, Interfaces and Ports
- ▲ **Future System C3.0**
 - ▲ Modeling of OSs
 - ▲ Support of embedded S/W models

Que peut on faire avec SystemC ?

- ⊙ A l'heure actuelle, SystemC propose une technologie mature permettant de modéliser des systèmes complexes composés de parties logicielles et matérielles,
 - Technique de gestion de processus (fork, join)
 - Technique de gestion de canaux abstraits (Transfer Level Modeling)
 - Technique de gestion des données au bit près (entières, flottantes, virgule fixe)
 - Bibliothèque de vérification (pour tester les systèmes),
- ⊙ Le calendrier prévisionnel n'a pas été respecté mais le langage évolue tout en remplissant ses objectifs
- ⊙ L'outillage professionnel développé autour de SystemC présente des lacunes surtout à bas niveau (synthèse logique par exemple)

Points forts de SystemC

- Support Hardware-Software Co-Design
- Interface in a C++ environment
- Modules and ports
- Container class includes hierarchical Entity and Processes
- Processes Describe functionality, Event sensitivity
- Single-directional (in, out), Bi-directional (inout) mode
- Signals : Resolved, Unresolved signals
- Rich set of port and signal types
- Rich set of data types
- All C/C++ types, 32/64-bit signed/unsigned, fixed-points, MVL, user defined

Caractéristiques de SystemC

- Clocks
- Special signal, Timekeeper of simulation and Multiple clocks, with arbitrary phase relationship
- Cycle-based simulation
- High-Speed Cycle-Based simulation kernel
- Multiple abstraction levels
- from high-level functional model to detailed clock cycle accuracy RTL model
- Communication Protocols
- Debugging Supports
- Run-Time error check
- Waveform Tracing

Point de départ

Every C/C++ program has a **main()** function. When the SystemC library is used, the **main()** function is not used. Instead the function **sc_main()** is the entry point of the application.

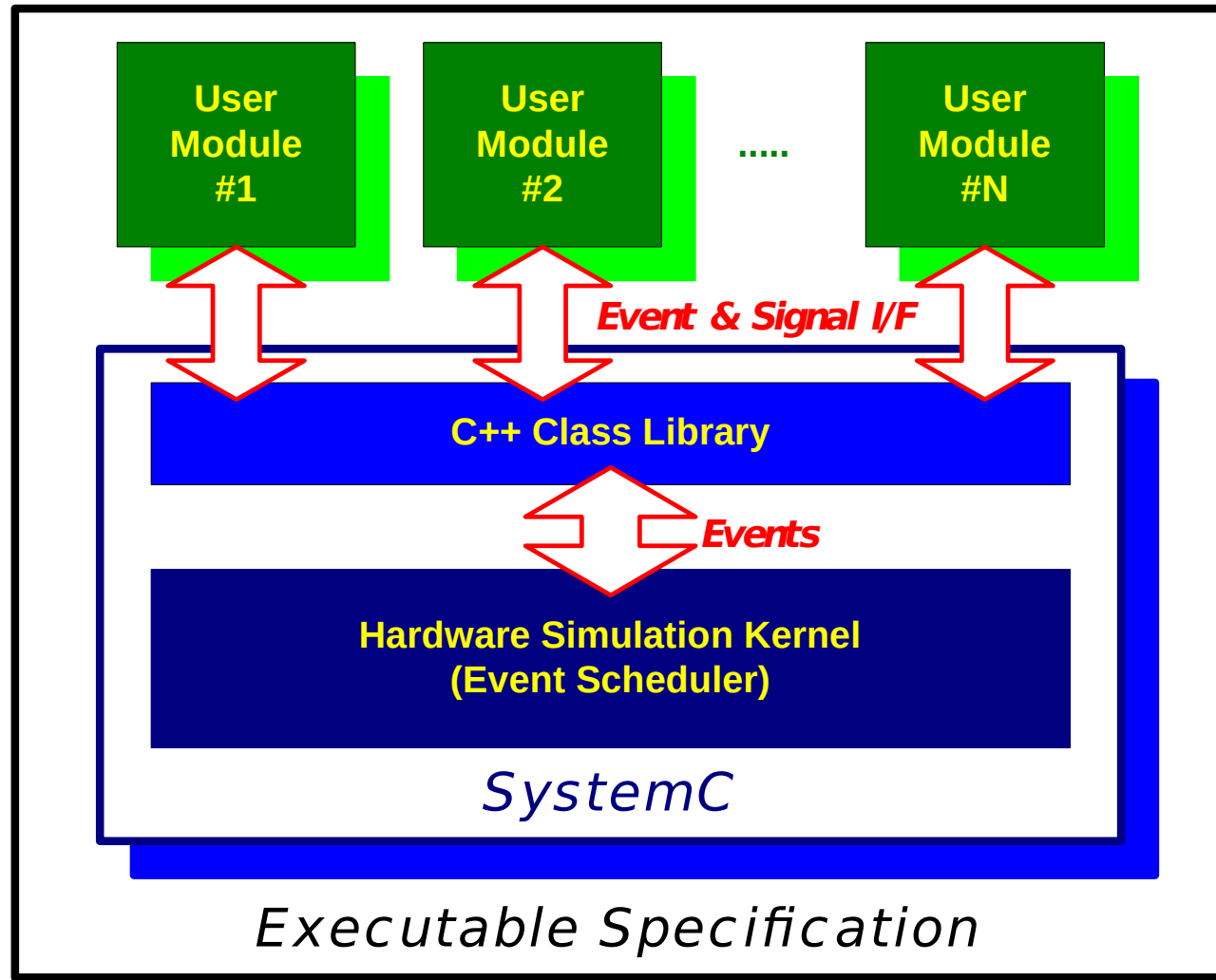
The **main()** function is defined into the library so that when the program starts initialization of the simulation kernel and the structures this requires to be performed, before execution of the application code.

In the **sc_main()** function the structural elements of the system are created and connected throughout the system hierarchy.

Access to all SystemC classes and functions is provided in a single header file named `systemc.h`

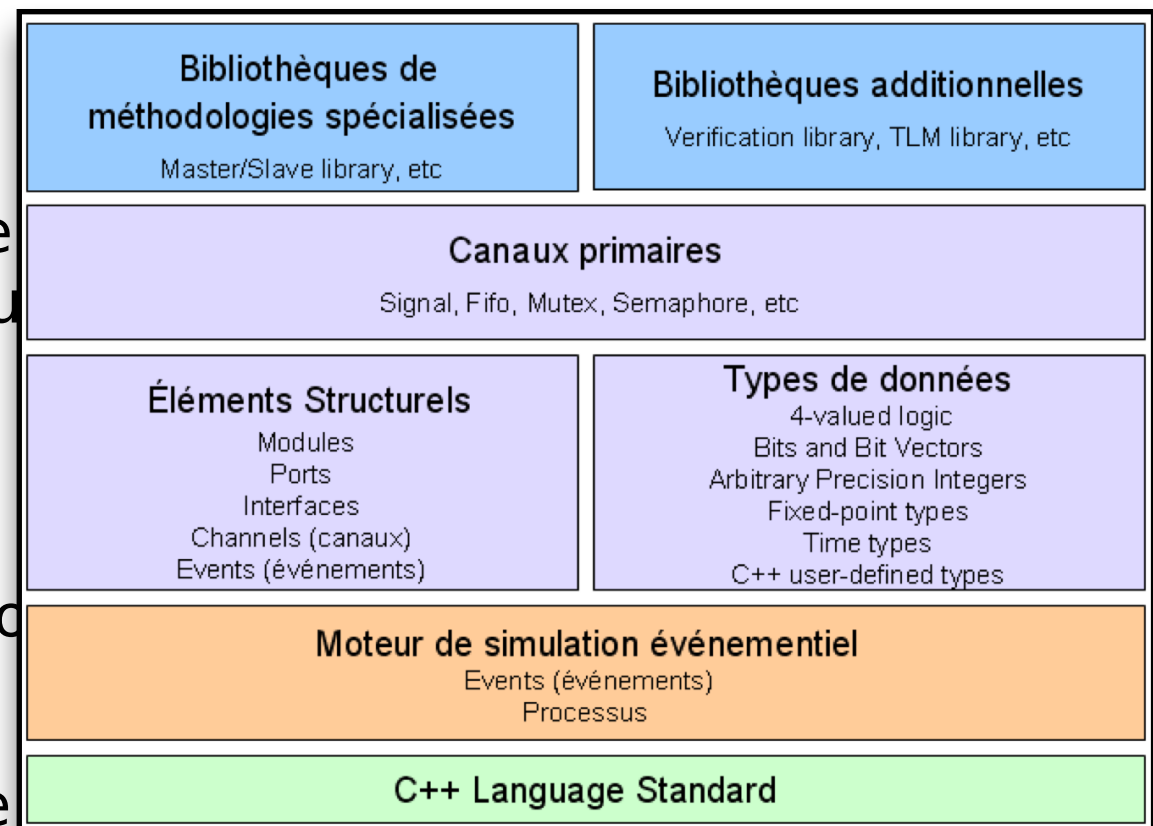
SystemC system consists of a set of one or more modules.

Modules SystemC/utilisateur

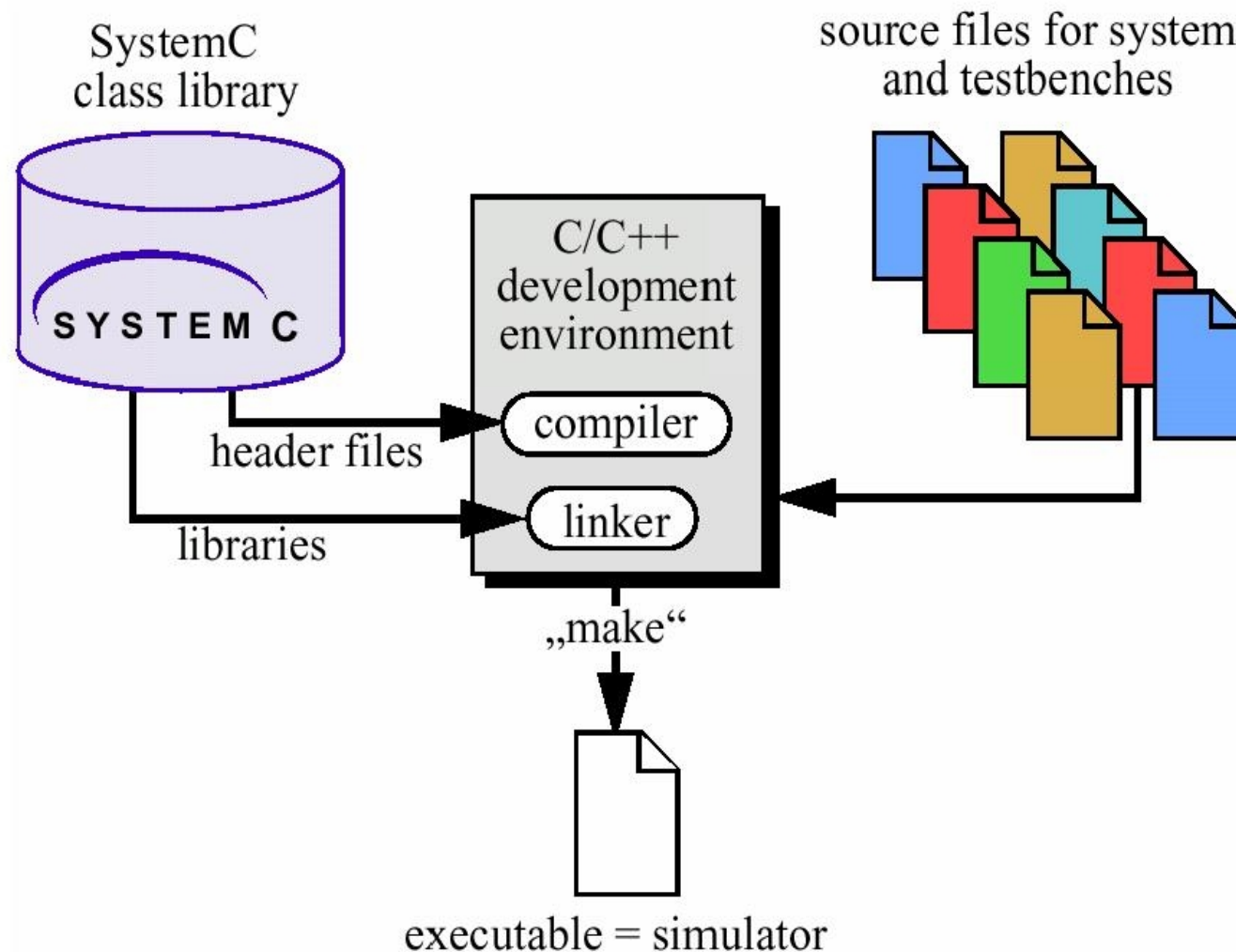


Hiérarchie des classes SystemC

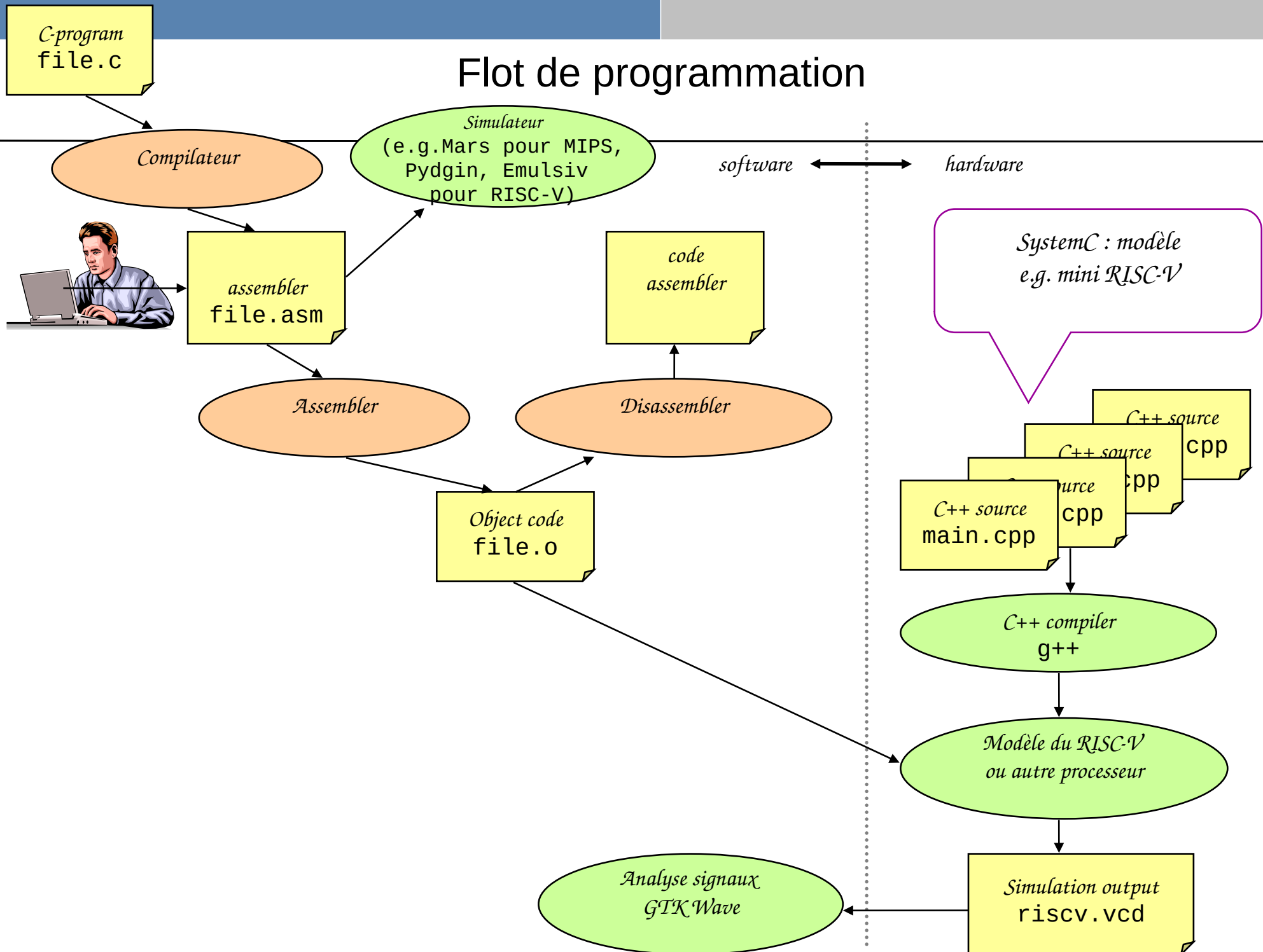
- SystemC est une bibliothèque de classes qui permet de modéliser le comportement de blocs logiciels & matériels
- SystemC est un ensemble briques de base utiles pour modéliser un système,
Modéliser des données typées, des canaux de communication (bus, fifo, mutex), etc...
- SystemC intègre d'origine moteur de simulation événementiel



Flot de conception SystemC



Flot de programmation



Les modules

- ⊙ Un **module** en SystemC est composé de **ports (E/S)**, de **canaux de communication** ainsi que de **processus**

- ⊙ Exemples

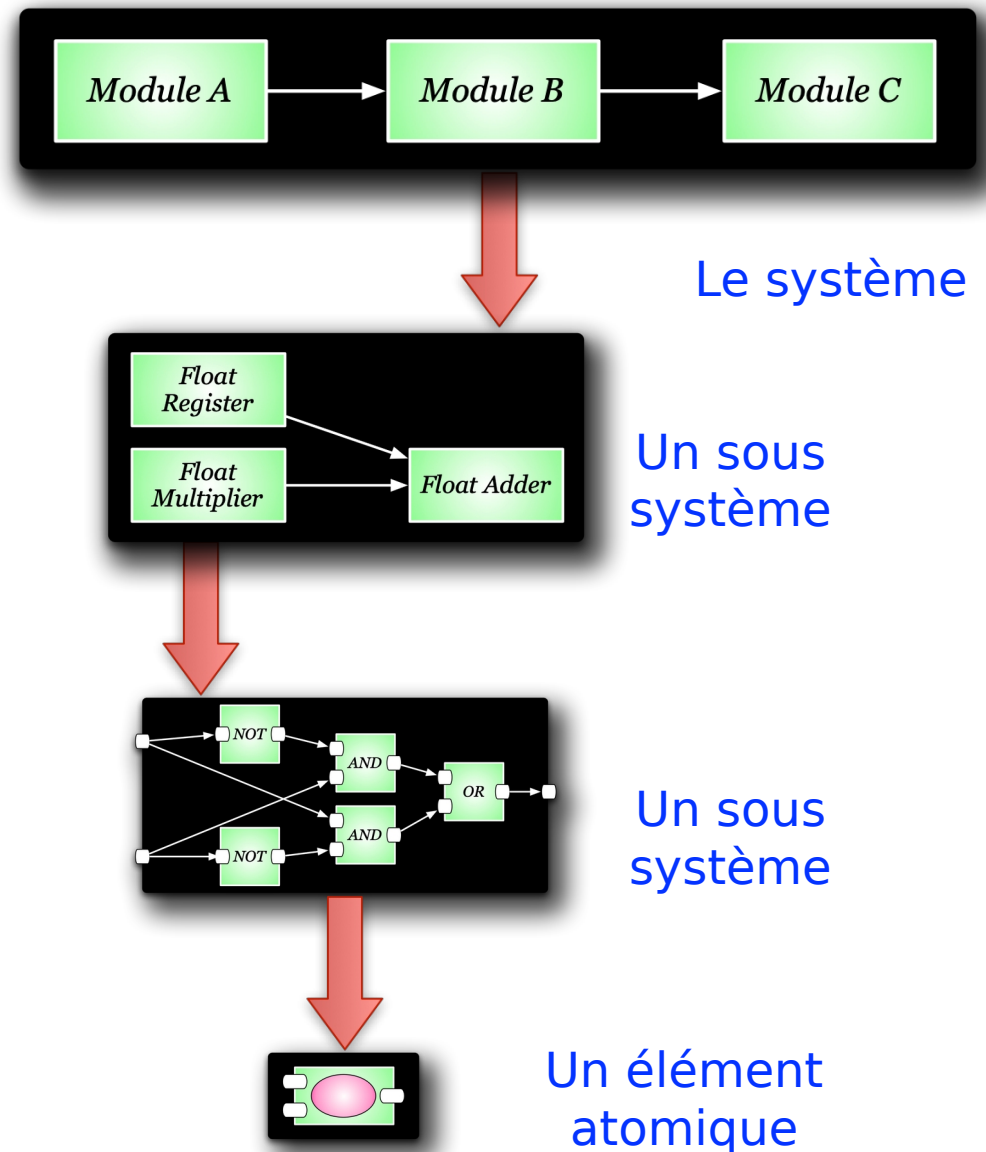
Un *processeur* est un module (qui contient éventuellement d'autres sous-modules : registres, ...)

Les portes AND, OR, NOT

Une mémoire RAM

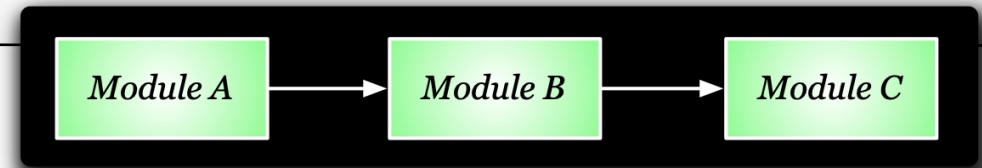
Un réseau (NoC, Internet, etc.)

Une caméra

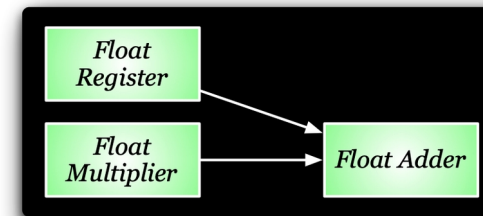
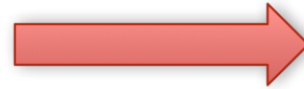


Les modules

Le système

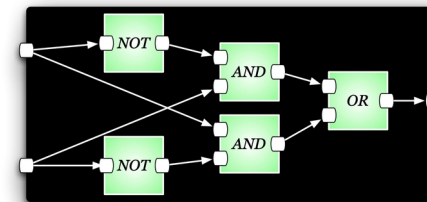
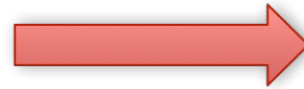


Un module peut être défini à partir d'une approche mixte (comportement + structurelle)



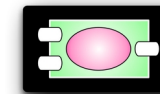
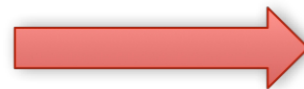
Un sous système

Un module peut être défini à partir d'une structure d'éléments défini ailleurs (boite noire)



Un sous système

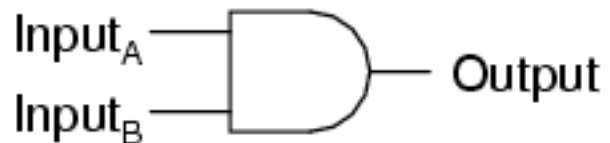
Un module peut être défini à partir de son comportement



Un élément atomique

Premier contact avec le langage SystemC

2-input AND gate



A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

```
void Porte_AND::do_and(){  
    s = a && b;  
}
```

Code source de la porte AND

```
#include "systemc.h"  
  
SC_MODULE(Porte_AND)  
{  
    sc_in<bool> a, b;  
    sc_out<bool> s;  
  
    SC_CTOR(Porte_AND)  
    {  
        SC_METHOD(do_and);  
        sensitive << a << b;  
    }  
  
    void do_and();  
};
```

Description du comportement

Comment définit on un module en SystemC ?

- ⊙ Le langage SystemC est une extension du C++

Les modules sont définis sous la forme de classes C++,

- ⊙ Les modules possèdent

Une définition (type classe)

Un constructeur

Un destructeur

Des attributs et méthodes

- ⊙ Afin de simplifier la vie des concepteurs, des macros ont été définies.

La déclaration de la classe

```
#include "systemc.h"

class module_un : public sc_module{
private:
    // La zone privée

public:
    module_un(sc_module_name name) : sc_module(name){
        cout << "Constructeur 1" << endl;
    }

    SC_HAS_PROCESS(module_un);

    ~module_un( ){
        cout << "Destructeur" << endl;
    }
};
```

Le constructeur

Le destructeur

Une simplification de la syntaxe du langage C++

Description objet laborieuse. Les constructeurs doivent toujours appeler le constructeur du parent.

```
#include "systemc.h"

class module_un : public sc_module{
private:
    // La zone privée

public:
    module_un(sc_module_name name) : sc_module(name){
        cout << "Constructeur 1" << endl;
    }

    SC_HAS_PROCESS(module_un);

    ~module_un(){
        cout << "Destructeur" << endl;
    }
};
```

L'utilisation des macros définies par SystemC permet de simplifier l'écriture du code sources des modules.

```
SC_MODULE(module_deux)
{
private:
    // La zone privée

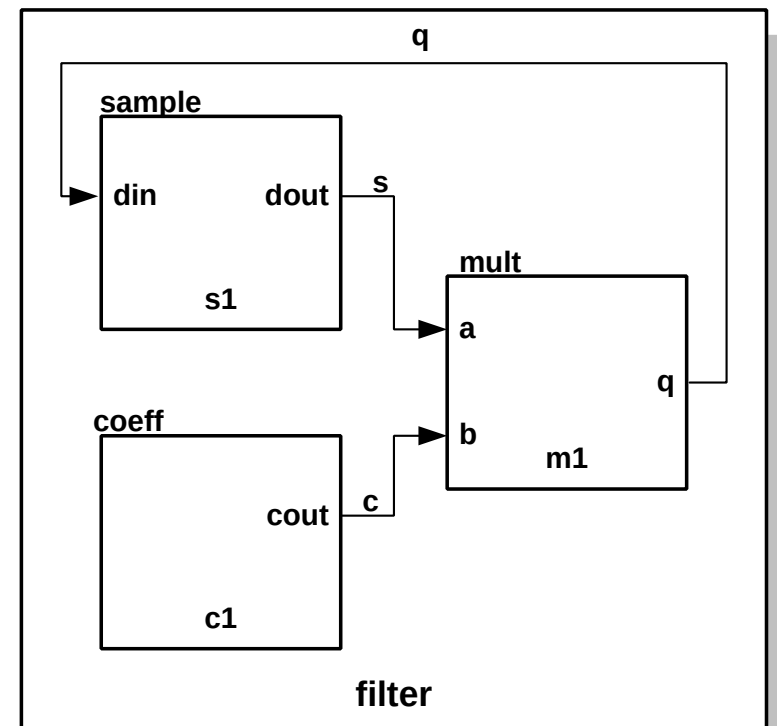
public:
    SC_CTOR(module_deux)
    {
    }

    ~module_deux(){
        cout << "Destructeur" << endl;
    }
};
```

Sous-modules et connections

```
SC_MODULE(filter) {  
    // Sub-modules : "components"  
    sample *s1;  
    coeff *c1;  
    mult *m1;  
  
    sc_signal<sc_uint 32> > q, s, c; // Signals  
  
    // Constructor : "architecture"  
    SC_CTOR(filter) {  
        // Sub-modules instantiation and mapping  
        s1 = new sample ("s1");  
        s1->din(q);      // named mapping  
        s1->dout(s);  
  
        c1 = new coeff("c1");  
        c1->out(c);      // named mapping  
  
        m1 = new mult ("m1");  
        (*m1)(s, c, q); // Positional mapping  
    }  
}
```

Exemple: filtre



Templates

- Exemple:

```
template <int W>
class sc_bv
    : public sc_bv_base
{
public:
    sc_bv();

    lrotate( int n );
    set_bit(int i, bool value);
    ...
}
```

Mot W comme paramètre

```
void main(void) {
    sc_signal< sc_bv<32> > bus_mux1;
}
```

Signal wires

Vecteur 32 bits

Templates

Fonctions similaires mais utilisant des types de donnée différents

```
short maximum (short a, short b) {  
    if(a > b)  
        return a;  
    else  
        return b;  
}  
  
int maximum (int a, int b) {  
    if(a > b)  
        return a;  
    else  
        return b;  
}  
  
double maximum (double a, double b) {  
    if(a > b)  
        return a;  
    else  
        return b;  
}
```

```
void main(void) {  
    double p = 10.0, q = 12.0;  
    int r = 15, s = 1;  
  
    double a = maximum(p, q);  
    int b = maximum(r, s);  
}
```

Eviter la duplication en faisant
du type un paramètre

Templates

Declares T as a 'variable' type

```
template <class T>
T maximum (T a, T b) {
    if(a > b)
        return a;
    else
        return b;
}
```

returns type T

a and b are of type T

```
void main(void) {
    double p = 10.0, q = 12.0;
    int r = 15, s = 1;

    double a = maximum(p, q);
    int b = maximum(r, s);
}
```

Uses the integer type

- Behind the scenes, the compiler builds the routine for each class.
- This is a little heavy on the compiler, and also harder to debug.

Templates

- Idem pour des classes

```
template <class T>
class coordinate {
public:
    coordinate(T x, T y) { _x = x; _y = y; }
    ~coordinate();

    void print(void) {
        cout << x << " , " << y << endl;
    }
private:
    T _x, _y;
}
```

The class datamembers
_x and _y of parameterized type T

```
void main(void) {
    coordinate <int> a(1, 2);
    coordinate <double> b(3.2, 6.4);

    a.print();
    b.print();
}
```

1 , 2
3.2 , 6.4

b is the double incarnation
of coordinate.

Again, the compiler builds a separate code instance for each type

Les ports des modules

- ⊙ Les **entrées-sorties** des modules sont appelées des ports.

Les ports sont déclarés dans les classes comme des **attributs publics**.

- ⊙ Les ports sont des éléments doublement typés,

Un type spécifie le sens du port de communication, entrée du module (**sc_in<T>**), sortie du module (**sc_out<T>**) ou entrée et sortie (**sc_inout<T>**)

Un type (ou paramètres) va spécifier la nature du port de communication qui va interagir avec l'extérieur (int, char*, sc_int, sc_bigunit, ma_structure, ...)

- ⊙ Exemples

L'entrée d'horloge d'un processeur est un port d'entrée (sc_in), appelé par ex. clk. Elle sera reliée à un signal normal (un fil) de type booléen (bool)

▸ **sc_in<bool> clk;**

Le bus d'adresse d'un processeur 32bits nommé mon_bus est un port de sortie (sc_out). Il pourra être déclaré comme suit:

▸ **sc_out<sc_lv<32> > mon_bus**

▸ **sc_out<sc_uint<32> > mon_bus**

Modélisation des valeurs logiques (std_logic_vector)

- ⊙ Le langage SystemC intègre des classes permettant des types de données plus proches du matériel.

sc_bit

- Cette classe permet de modéliser les bits qui peuvent prendre la valeur 1 signifiant true et 0 pour false.

sc_logic

- Cette classe permet de modéliser les valeurs logiques (bits) qui nécessitent les 4 états : {0, 1, X, Z} que ne gère pas intégralement sc_bit.

sc_bv<N>

- Cette classe permet de modéliser un vecteur de bits (éléments de type sc_bit) au sein d'un même objet,

sc_lv<N>

- Cette classe permet de modéliser un ensemble d'éléments de type sc_lv au sein d'un même objet,

Equivalences entre types d'E/S en VHDL et en SystemC

VHDL Port	SystemC Port	SystemC Notes
in	sc_in	
out	sc_out	Behaves like a VHDL buffer port
inout	sc_inout	
buffer	sc_inout	

VHDL	SystemC	SystemC Notes
bit	sc_bit	Use native C++ bool type instead
bit_vector	sc_bv	Faster in simulation than sc_lv
std_ulogic	sc_logic	Only support 4 types, 'X','Z','0' and '1'
std_ulogic_vector	sc_lv	Only support 4 types, 'X','Z','0' and '1'
std_logic	sc_logic_resolved	Resolution function for 'X','Z','0' and '1'
std_logic_vector	sc_signal_rv	Resolution function for 'X','Z','0' and '1'
boolean	bool	Recommended instead of sc_bit

http://www.ht-lab.com/howto/vh2sc_tut/vh2sc_tut2.html

Valeurs numériques

Equivalences entre VHDL et SystemC (2)

VHDL	SystemC	SystemC Notes
boolean	bool	Recommended instead of sc_bit
unsigned	sc_uint<N>	N bits unsigned integer vector (N <=64)
	sc_biguint<M>	Arbitrary length unsigned integer vector
signed	sc_int<N>	N bits signed integer vector (N <=64)
	sc_bigint<M>	Arbitrary length signed integer vector
integer	int	int size is platform dependent, use sizeof(int)
	sc_int<N>	N bits signed integer vector (N <=64)
	sc_bigint<M>	Arbitrary length signed integer vector
real	float	Simulation purpose only, Native C/C++ 32bits floating point number

http://www.ht-lab.com/howto/vh2sc_tut/vh2sc_tut2.html

Modélisation des valeurs logiques (std_logic_vector)

```
sc_bit a, b;  
bool c;  
a = '1';  
b = false;  
a &= b;  
if( c || a) {...}  
  
sc_logic out;  
bool data, enable;  
  
if(enable)  
    out = data;  
else  
    out = 'Z';
```

```
sc_bv<16> data16;  
sc_bv<16> data15;  
sc_bv<8> x;  
sc_bit read_ok;  
  
x = "01000111";  
  
data32.range(14,0) = data15;  
data32.range(15,15) = data15.xor_reduce();
```

Modélisation des valeurs numériques (signed/unsigned)

- ⊙ Le langage SystemC intègre des classes permettant de modéliser des calculs avec une définition proche du matériel.
- ⊙ Modélisation des “nombres entiers” (1 => 64 bits)
 - `sc_int<T>`, classe modélisant des nombres entiers positifs ou nuls de précision figée à l’instanciation de l’objet (T).
 - `sc_uint<T>`, classe modélisant des nombres entiers positifs ou négatifs de précision figée à l’instanciation de l’objet (T).
- ⊙ Modélisation de grands nombres (<MAX_NBITS bits)
 - `sc_bigint<T>`, classe modélisant des nombres entiers positifs ou nuls de variant jusqu’à 512 bits. La précision souhaitée est spécifiée à l’instanciation de l’objet (T).
 - `sc_bignint<T>`, classe modélisant des nombres entiers positifs ou négatifs variant jusqu’à 512 bits. La précision souhaitée est spécifiée à l’instanciation de l’objet (T).

- ⊙ Les classes associées à la modélisation des nombres entiers proposent les méthodes suivantes :

Constructeurs adaptés à la création d'objets avec ou sans données à initialiser

Sélection de bit à l'aide des opérateurs "[]"

Test de la valeur d'un bit contenu dans le vecteur `bool test(int)`

Forçage la valeur d'un bit contenu dans le vecteur `set(int, bool)`

Récupérer la **largeur** en nombre de bit de la donnée `int length( )`

Sélection d'un ensemble de bits par la méthode `range(int, int)`

Concaténation de bits avec l'opérateur `" "` ou la méthode `concat( )`

- ⊙ SystemC impose le MSB à gauche (n-1) et le LSB à droite (0).

Manipulation des valeurs numériques (signées/on signées)

```
sc_uint <32> input, output; // entier non signé sur 32 bits
sc_uint <1>  sign;           // entier non signé sur 1  bit
sc_uint <7>  expo;           // entier non signé sur 7  bits
sc_uint <24> mant;           // entier non signé sur 24 bits

sign = input.test(32);
// sign[0] = input[32];

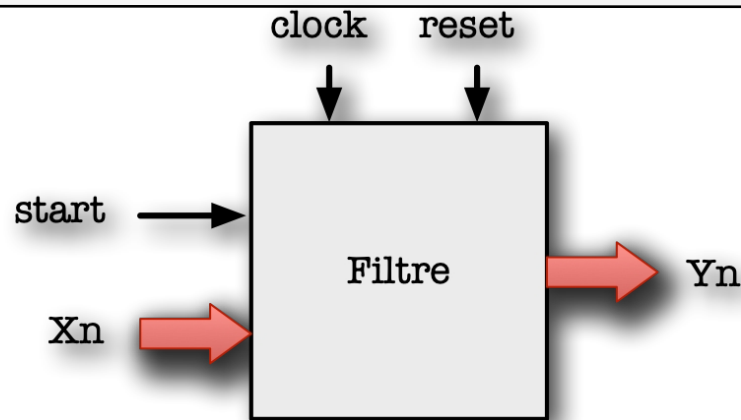
mant = input.range(23, 0);
expo = input.range(30, 24);

sign.set( ! sign );

expo = expo + 1;

output = (sign, expo, mant);
```

Exemples de déclarations en SystemC et en VHDL



Les déclarations d'un filtre FIR en VHDL et en SystemC sont similaires au langage près.

```
ENTITY Filtre is
  PORT (
    CLOCK : IN  STD_LOGIC;
    RESET  : IN  STD_LOGIC;
    START  : IN  STD_LOGIC;
    XN     : IN  STD_LOGIC_VECTOR(15 DOWNTO 0);
    YN     : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
  );
END Filtre;
```

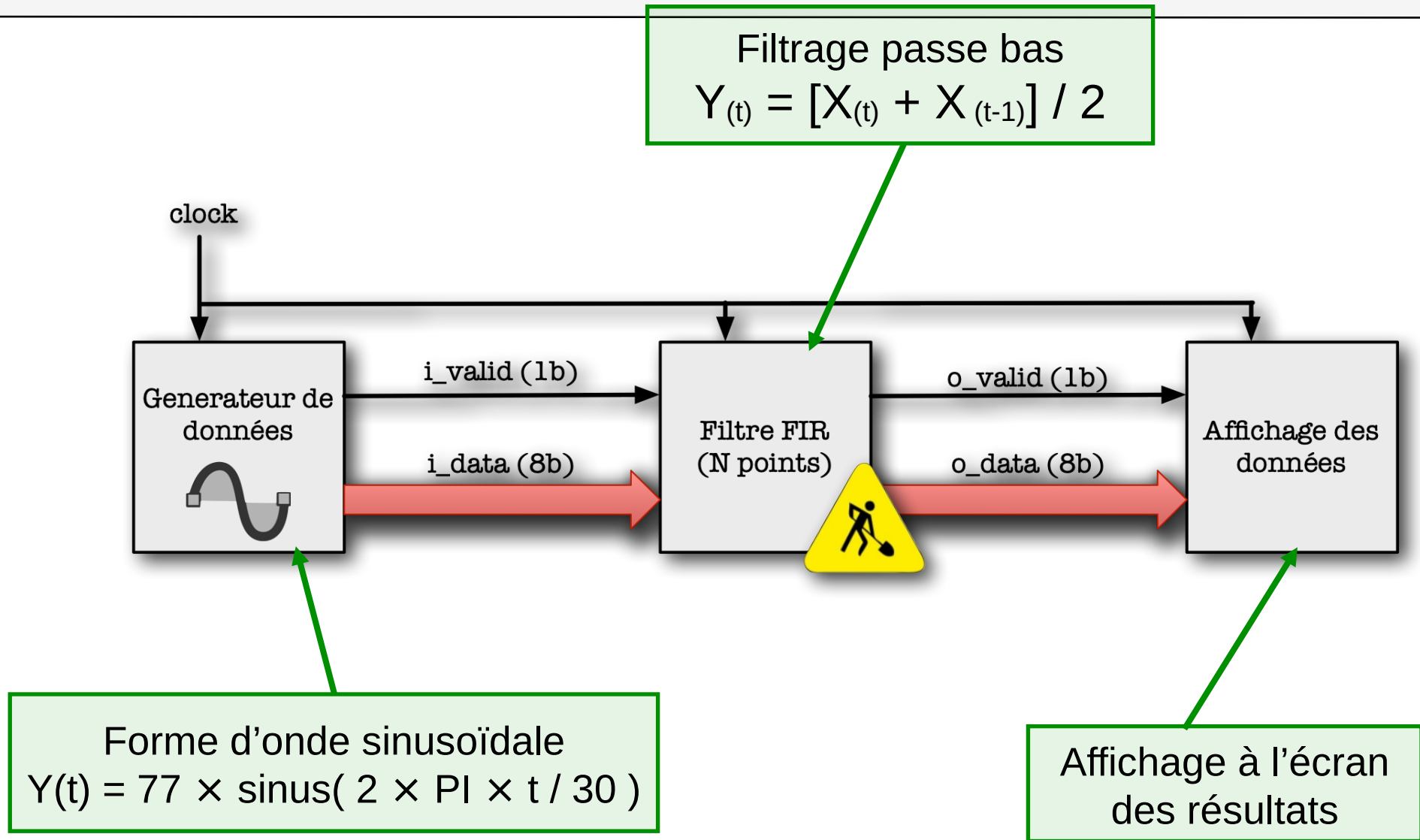
Déclaration en VHDL

```
SC_MODULE(Filtre)
{
  public:
    sc_in <bool>    clk;
    sc_in <bool>    reset;
    sc_in <bool>    start;
    sc_in < sc_lv <16> > Xn;
    sc_out< sc_lv <16> > Yn;

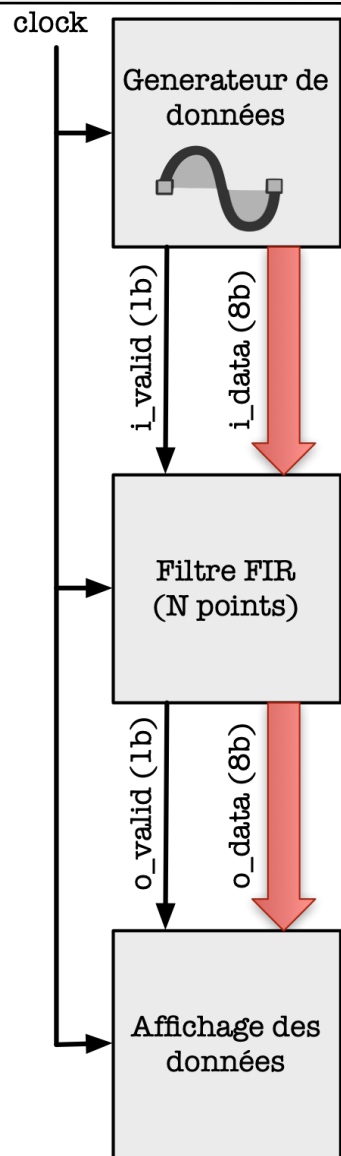
    SC_CTOR(Filtre){
    }
};
```

Déclaration en SystemC

Exemple: description d'un système (VHDL versus SystemC)



Exemple pédagogique (VHDL)

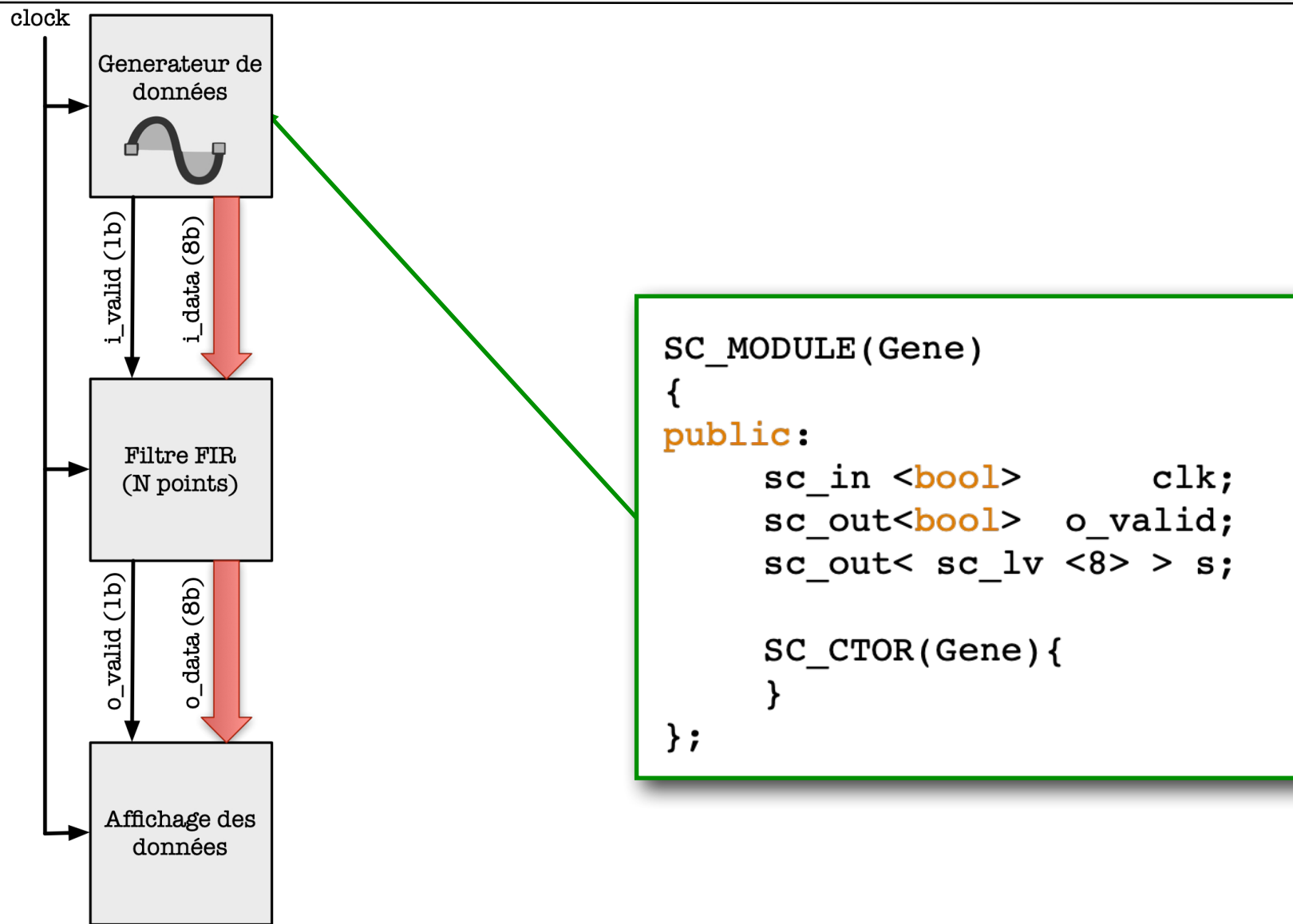


```
ENTITY Generator IS
  PORT (
    CLOCK   : IN  STD_LOGIC;
    RESET   : IN  STD_LOGIC;
    OUTPUT  : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
    O_VALID : OUT STD_LOGIC
  );
END Generator;
```

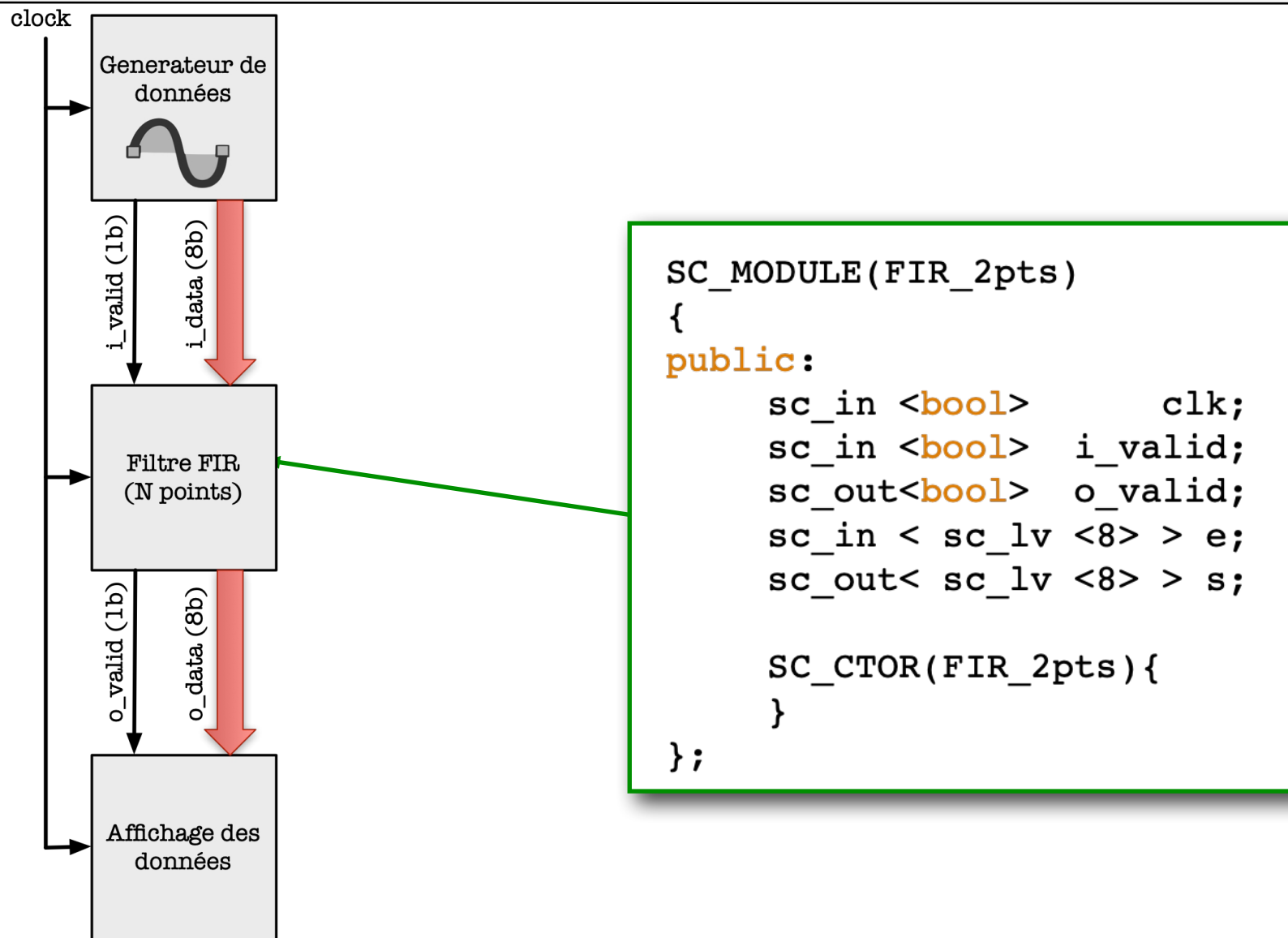
```
ENTITY Filtre is
  PORT (
    CLOCK   : IN  STD_LOGIC;
    RESET   : IN  STD_LOGIC;
    INPUT   : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
    I_VALID : IN  STD_LOGIC;
    OUTPUT  : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
    O_VALID : OUT STD_LOGIC
  );
END Filtre;
```

```
ENTITY Afficheur is
  PORT (
    CLOCK   : IN  STD_LOGIC;
    RESET   : IN  STD_LOGIC;
    INPUT   : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
    I_VALID : IN  STD_LOGIC
  );
END Afficheur;
```

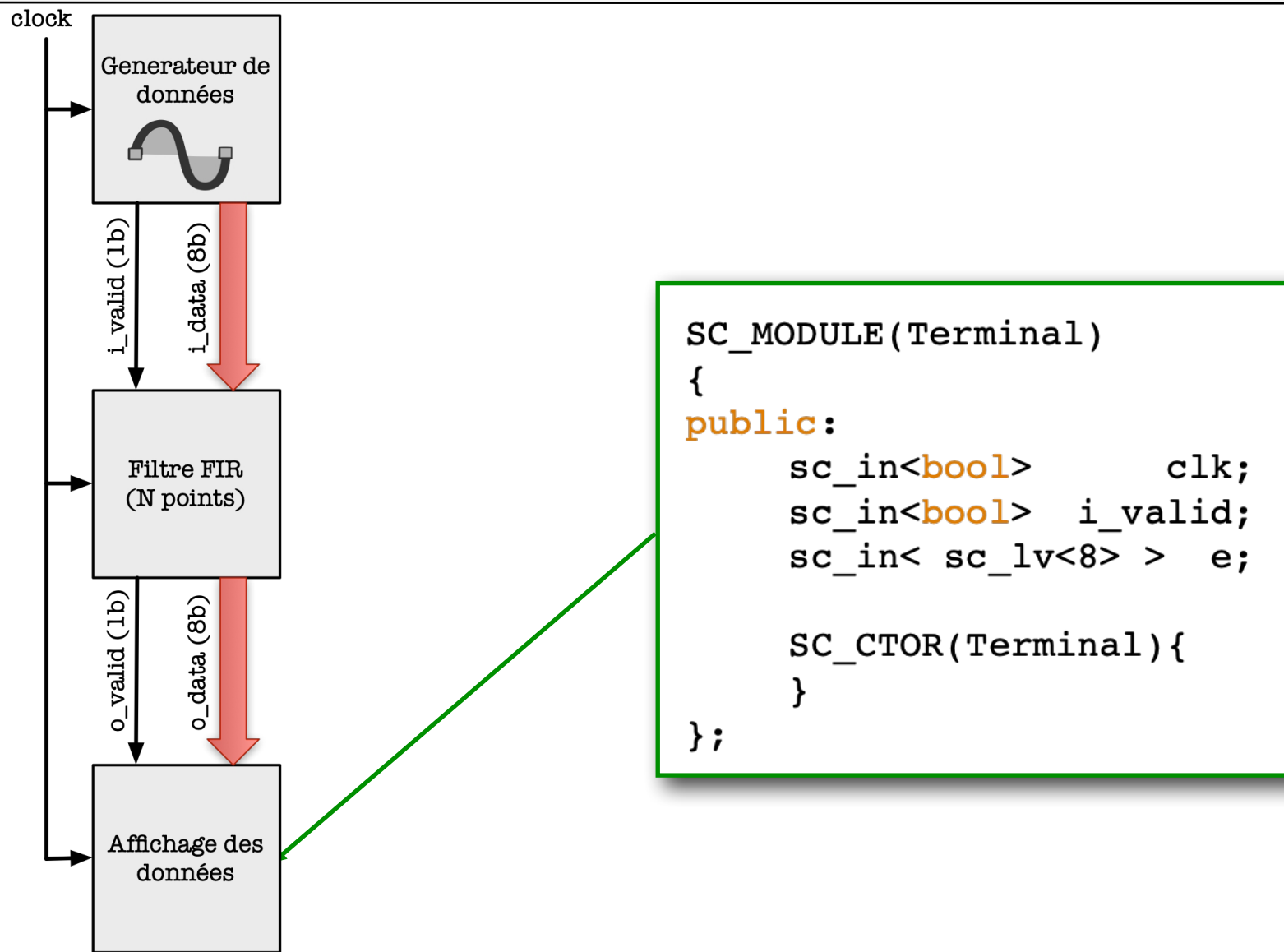
Exemple pédagogique (SystemC) - Module de filtrage



Exemple pédagogique (SystemC) - Module de génération



Exemple pédagogique (SystemC) - Module d'affichage



Modélisation du comportement : Les processus

- ⊙ Les processus sont des méthodes qui décrivent toute ou partie du comportement d'un module.
- ⊙ Un processus (la fonction) ne doit jamais être appelé directement
Le moteur de simulation SystemC se charge d'exécuter la fonction lorsqu'un événement relatif signaux d'entrée du processus sont détectés (changement des entrées implique peut être un changement des sorties ?)

⊙ Exemples

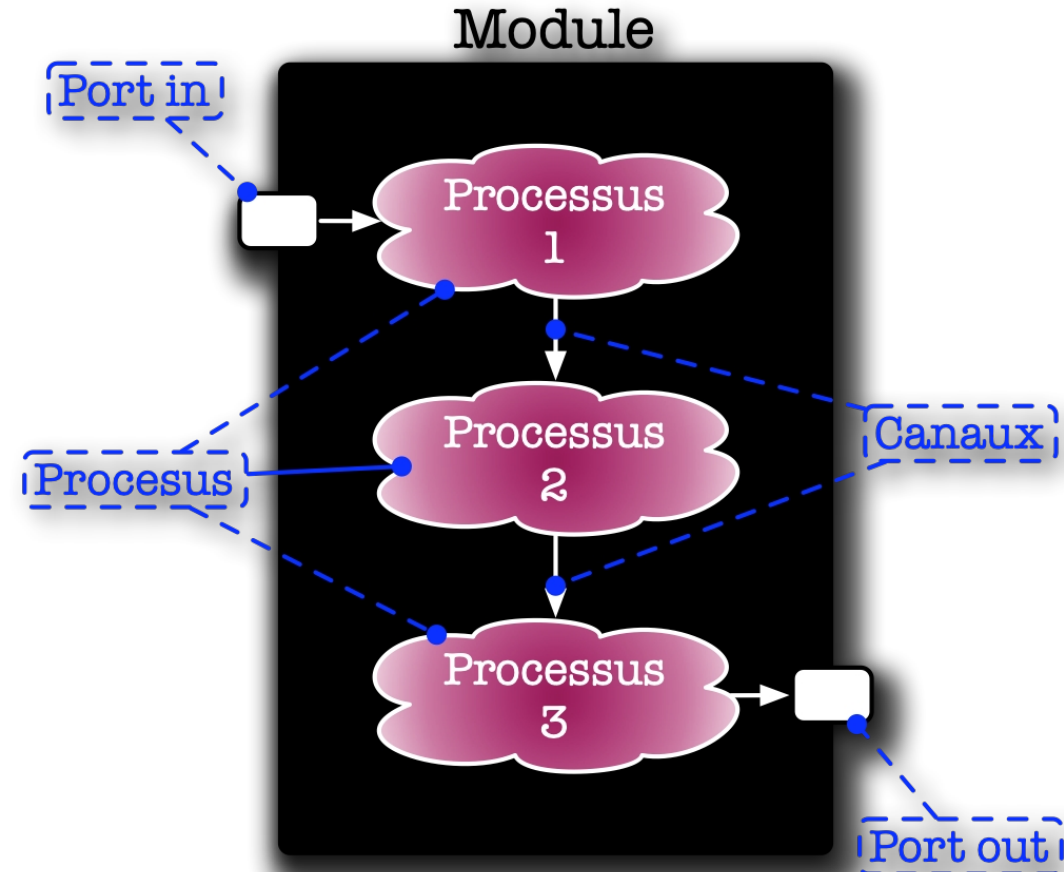
Une porte ET doit re-calculer sa sortie dès que l'une de ses entrées change. La fonction qui calcule le comportement (ET) est modélisée à l'aide d'un processus réactif sur les entrées de la porte. Lorsqu'une entrée est modifiée, le processus de calcul est exécuté automatiquement par le simulateur et la sortie mise à jour.

Un registre n'est mis à jour que sur front montant de l'horloge. Le processus chargé du stockage de l'entrée (et de la mise à jour de la sortie) est réactif uniquement sur l'horloge qui doit être dans sa liste de sensibilité.

Un composant de compression vidéo type MPEG doit être "exécuté" uniquement lorsqu'une nouvelle image est disponible et à condition qu'il ait terminé de traiter la précédente.

Introduction aux processus

- ⊙ Les processus constituent le coeur des modules. Ils sont en charge de la description du comportement du composant,
- ⊙ Un processus est défini comme étant une méthode,
A la création de l'objet, on va spécifier au simulateur quelles méthodes doivent être exécutées en cas d'apparition de stimulus externes,
- ⊙ Les processus vont communiquer entre eux à l'aide de canaux de communication.



Gestion de ces différents comportements

- ⊙ SystemC met à la disposition du concepteur 3 types de processus possédant des propriétés différentes

SC_METHOD

- Les processus de ce type sont privilégiés pour les composants dont le comportement est déclenché à chaque changement d'une de ses entrées. Ce type est souvent utilisé pour modéliser les composants combinatoires (asynchrones)

SC_THREAD

- Les processus de ce type sont mis en oeuvre pour gérer les comportements synchrones ou la liste de sensibilité peu évoluer dans le temps (FSM dont les actions provoquant des transitions varient en fonction de l'état courant)

SC_CTHREAD

- Les processus de ce type correspondent à un sous-ensemble des SC_THREAD. Ces processus évoluent uniquement sur les fronts d'horloge (sensibilité unique).

- ⊙ Le choix du type de modélisation est guidé par le type de comportement à modéliser.

Le constructeur du module: liste de sensibilité du processus

- **Processus** disposent d'une liste d'évènements auxquels ils sont "réactifs"
- `sensitive(rwb)` crée cette liste pour le processus enregistré par le précédent `SC_method`
- Le processus en question s'exécutera quand un événement est déclenché par le port `rwb`
- Les ports déclenchent d'évènements via le channel auquel ils sont connectés si la valeur sur ce port change ou si la valeur sur un autre port connecté au même channel change

Un processus est instancié en 4 étapes

⊙ Déclaration du processus

Les processus sont des méthodes de la classe définissant le comportement temporel du module. Comme toute méthode il doit avoir une déclaration du prototype (.h) et une implémentation du coeur de la méthode (.cpp)

⊙ Enregistrement

Les processus (méthodes) ne sont pas appelés directement, c'est le simulateur SystemC qui les appelle ou réveille lorsqu'il reçoit un signal sur lequel ils sont sensibles

Le simulateur doit tout d'abord savoir quelles méthodes sont des processus ainsi que la catégorie de processus associé.

⊙ Déclaration de la liste de sensibilité par défaut

Le simulateur événementiel à besoin de savoir quelles sont les actions qui doivent réveiller chacun des processus déclarés précédemment..

⊙ Implémentation

Comme pour toutes méthodes, il faut fournir une implémentation de la méthode, généralement réalisée dans le fichier (cpp) associé à la classe.

Le fonctionnement des SC_METHOD

⊙ Introduction

Les **SC_METHOD** sont des processus particuliers qui permettent de modéliser le comportement de composants dont la liste de sensibilité est figée (statique) lors de l'exécution de la simulation,

Les **SC_METHOD** ne sont pas des threads à part entière car ils doivent rendre la main après chaque exécution (return;) sinon cela bloque la simulation !

- ▶ Un processus de type **SC_METHOD** est d'un point de vue sémantique une simple fonction (notion de processus en VHDL)

Les processus de classe **SC_METHOD** sont lancés à chacun des changements intervenus sur leur liste de sensibilité (A puis B => 2 exécutions),

⊙ Remarques

Les variables locales contenues dans des méthodes de type **SC_METHOD** sont ré-initialisées à chaque entrée dans le processus.

Si des données doivent être mémorisées, il faut le faire à l'aide d'attributs dans la classe (variables globales dans ce cas de figure).

Gestion de la liste de sensibilité du processus

- ⊙ La spécification de la liste de sensibilité se fait dans le constructeur du module (après la déclaration du processus et de son type),

Pour ajouter un signal dans la liste de sensibilité d'un processus, on se sert de la méthode `sensitive( )`. Cette méthode prend en argument un signal et s'applique à la dernière **SC_METHOD** enregistrée.

- ⊙ Il est possible de filtrer les fronts montants ou descendants (uniquement les signaux sur 1 bit),

Déclarer la **SC_METHOD** comme sensible à tous les fronts du signal (`sensitive << clock`), et discriminer dans le corps du processus les fronts (`if clock.posedge( ) { ... }`).

On peut aussi filtrer les fronts dans la liste de sensibilité à l'aide des constructions suivantes : `sensitive << clock.pos( )` et/ou `sensitive << mon_signal.neg( )`, ce qui accélère les simulations (moins d'exécution des processus).

- ⊙ Processus avec plusieurs sensibilités

```
sensitive << clock.pos() << clock.neg;
```

```
sensitive << Input_1 << Input_2;
```

Exemple de définition d'une SC_METHOD

```
SC_MODULE(mon_module) {  
  
    sc_in<bool> clk;           // Signaux arrivant en entrée de  
    sc_in<bool> reset;        // la classe  
  
    void comportement();      // Méthode contenant le comportement  
  
    SC_CTOR(mon_module) {     // Constructeur de la classe  
        SC_METHOD(ma_sc_method); // Enregistrement du processus  
        sensitive << enable;    // Insertion des signaux à gérer  
        sensitive(reset.pos()); // dans la liste de sensibilité  
        // ... ..  
    }  
}
```



On spécifie le type du processus ainsi que les signaux auxquels il est sensible

Les processus (modélisation séquentielle)

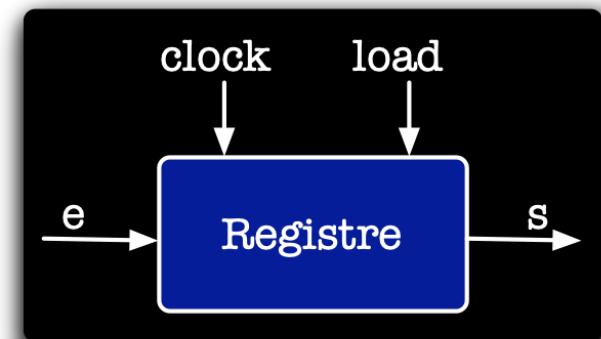
```
#include "systemc.h"

SC_MODULE(Registre)
{
private:
    sc_lv<8> lastValue;

public:
    sc_in <bool> clk;
    sc_in <bool> l;
    sc_in < sc_lv<8> > e;
    sc_out< sc_lv<8> > s;

    SC_CTOR(Registre)
    {
        SC_METHOD(do_reg);
        sensitive << clk.pos();
        lastValue = (sc_uint<8>)0;
    }

    void do_reg();
};
```



```
#include "Registre.h"

void Registre::do_reg(){
    if( l.read() == 1 ){
        s = e.read();
        lastValue = e;
    }else{
        s = lastValue;
    }
}
```

Les processus (modélisation séquentielle)

```
#include "systemc.h"
```

```
SC_MODULE(Registre)
```

```
{
```

```
public:
```

```
    sc_in<bool> clk;
```

```
    sc_in<bool> l;
```

```
    sc_in< sc_lv<8> > e;
```

```
    sc_out< sc_lv<8> > s;
```

```
    SC_CTOR(Registre)
```

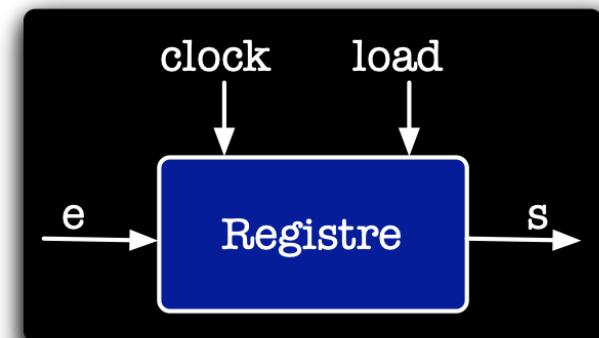
```
    {
```

```
        SC_THREAD(do_reg);
```

```
        sensitive << clk.pos();
```

```
    void do_reg();
```

```
};
```



```
#include "Registre.h"
```

```
void Registre::do_reg(){
```

```
    s = 0;
```

```
    while( true ){
```

```
        wait();
```

```
        if( l.read() == 1 ){
```

```
            s = e.read();
```

```
        }
```

```
    }
```

```
}
```

Les processus (modélisation séquentielle)

```
#include "systemc.h"
```

```
SC_MODULE(Registre)
```

```
{
```

```
public:
```

```
    sc_in <bool> clk;
```

```
    sc_in <bool> l;
```

```
    sc_in < sc_lv<8> > e;
```

```
    sc_out< sc_lv<8> > s;
```

```
    SC_CTOR(Registre)
```

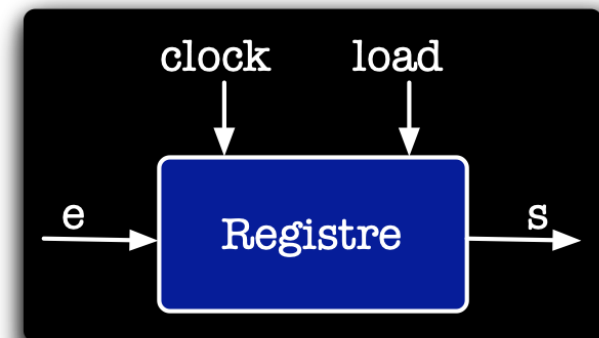
```
    {
```

```
        SC_CTHREAD(do_reg, clk.pos());
```

```
    }
```

```
    void do_reg();
```

```
};
```



```
#include "Registre.h"
```

```
void Registre::do_reg(){
```

```
    s = 0;
```

```
    while( true ){
```

```
        wait();
```

```
        if( l.read() == 1 ){
```

```
            s = e.read();
```

```
        }
```

```
    }
```

```
}
```

Les processus de type SC_THREAD

⊙ Introduction

Les SC_THREAD sont des processus particuliers implémentés sous forme de threads autonomes dans le simulateur.

Les SC_THREAD permettent de modéliser des processus dont la liste de sensibilité peut évoluer dynamiquement durant l'exécution.

Les SC_THREAD sont des processus développés autour d'une méthode généralement composée d'une boucle infinie qui s'endort en attente d'événements. Ces processus ne sont plus exécutés à chaque apparition d'un événement, ils sont réveillés !

⊙ Remarques

Une fois le traitement associé à un événement effectué, le SC_THREAD est mis en sommeil (attente d'un nouvel événement) à l'aide de la méthode wait().

Si un thread s'arrête en arrivant au bout du code à exécuter ou rencontre une instruction return alors il ne s'exécutera plus !

L'état des variables internes à un processus de type SC_THREAD sont conservées en l'état lors du réveil du processus car il est juste endormi...

SC_METHOD vs. SC_THREAD

- ⊙ SC_METHOD(func): does not have its own thread of execution, consumes no simulated time, cannot be suspended, and cannot call code that calls wait()
- ⊙ 2. SC_THREAD(func): has its own thread of execution, may consume simulated time, can be suspended, and can call code that calls wait() 3.
- ⊙ A simulation process:
 - ⊙ 1. is a member function of a sc_module class, and
 - ⊙ 3. has no input argument and returns no value, and
 - ⊙ 2. is registered with the simulation kernel
- ⊙ How to register a simulation process:
 - ⊙ 1. SC_METHOD(func): does not have its own thread of execution, consumes no simulated time, cannot be suspended, and cannot call code that calls wait()
 - ⊙ 2. SC_THREAD(func): has its own thread of execution, may consume simulated time, can be suspended, and can call code that calls wait()
 - ⊙ 3. SC_CTHREAD(func, event): a special form of SC_THREAD that can only have a static sensitivity of a clock edge event

SC_METHOD vs. SC_THREAD

- ⊙ When can registration happen:
 - ⊙ 1. in the body of the constructor,
 - ⊙ 2. in the before_end_of_elaboration or end_of_elaboration callbacks of a module,
 - ⊙ 3. or in a member function called from the constructor or callback
- ⊙ Restrictions:
 - ⊙ 1. registration can only be performed on member functions of the same module.
 - ⊙ 2. SC_CTHREAD shall not be invoked from the end_of_elaboration callback
- ⊙ Note:
 - ⊙ 1. SC_THREAD can do everything that SC_METHOD or SC_CTHREAD does. I'll mostly use this process in the examples.
 - ⊙ 2. In order for an SC_THREAD or SC_CTHREAD process to be called again, there shall be a while loop making sure it never exits.
 - ⊙ 3. An SC_THREAD process doesn't require a while loop. It is invoked again by next_trigger()
 - ⊙ 4. simulated time in systemC is not the actual time a program runs. It's a counter managed by the simulation kernel. To be explained later.

Mise en oeuvre des processus SC_THREAD

⊙ Création des SC_THREAD

Le processus de création des SC_THREAD est exactement identique à celui de création d'une SC_METHOD, en remplaçant cette dernière par la macro SC_THREAD...

La déclaration et la gestion de la liste de sensibilité est aussi identique.

⊙ Quand utiliser un SC_THREAD ?

Quand on a besoin d'introduire des notions de temps dans les modèles (attendre non plus un événement, mais un délai relatif avant d'entreprendre une action),

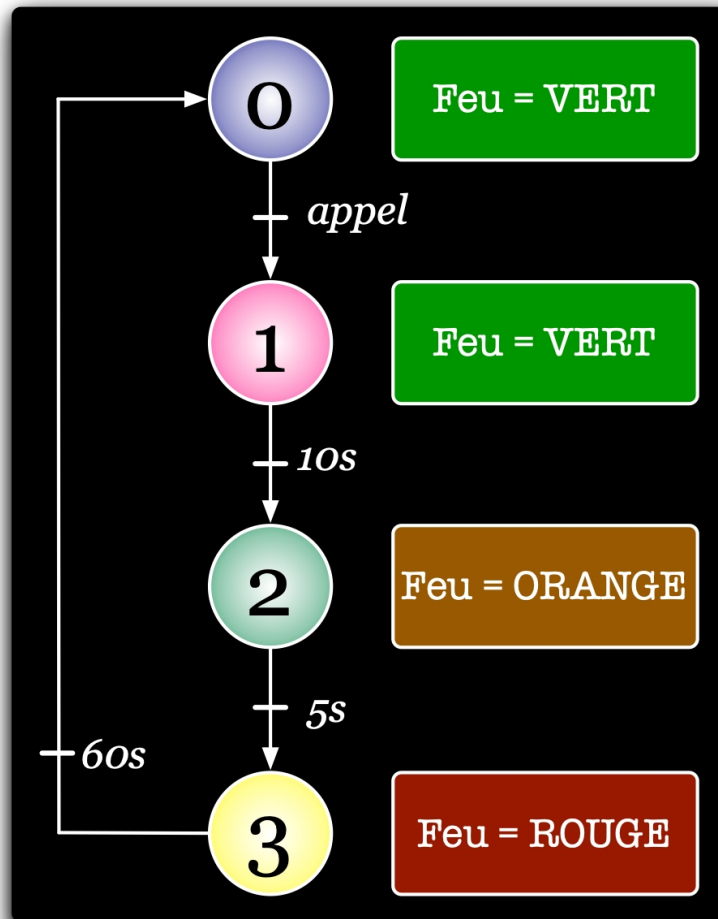
Modéliser des processus séquentiels, typiquement des machines synchrones ou asynchrones à états implicites (décrites étapes par étape).

Modéliser des processus qui stockent localement de l'information entre deux activations successives,

Modéliser des processus accédant à des ressources bloquantes (entrée-sorties, fichiers, Fifo, sémaphores, réseaux, ...),

Si on a le choix, les SC_METHOD qui simulent beaucoup plus rapidement.

Exemple d'un automate gérant un feu rouge



```
// Ce processus est un SC_THREAD possédant
// dans sa liste de sensibilité une hologe
// de fréquence 1Hz
void Gestion_Feu::mon_thread() {
    while ( true ) {
        couleur_feu = VERT;
        wait(appel_bouton);

        // Temporisation de 10 secondes
        // puis le feu doit passer ^ l'orange
        for(int i=0; i<10; i++) wait( );
        couleur_feu = ORANGE;

        // Temporisation de 5 secondes
        // puis le feu doit passer au rouge
        wait( 5, SC_S );
        couleur_feu = ROUGE;

        // Temporisation d'une minute
        // puis on repasse au vert (initial)
        wait(60, SC_SC);
    }
}
```

Les processus (modélisation combinatoire)

```
#include "systemc.h"
```

```
SC_MODULE(NAND_Gate)
```

```
{  
public:
```

```
    sc_in < bool > a;
```

```
    sc_in < bool > b;
```

```
    sc_out< bool > s;
```

```
    SC_CTOR(NAND_Gate)
```

```
    {
```

```
        SC_METHOD(do_nand);
```

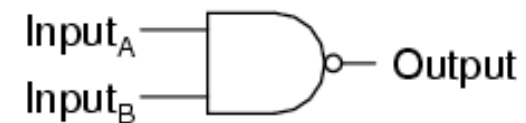
```
        sensitive << a << b;
```

```
    }
```

```
    void do_nand();
```

```
};
```

2-input NAND gate



A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

```
#include "NAND_Gate.h"
```

```
void NAND_Gate::do_nand() {
```

```
    s = !(a && b);
```

```
}
```

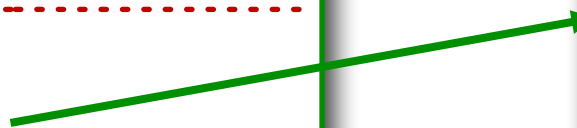
Les processus (modélisation combinatoire)

```
#include "systemc.h"

SC_MODULE(NAND_Gate)
{
public:
    sc_in < bool > a;
    sc_in < bool > b;
    sc_out< bool > s;

    SC_CTOR(NAND_Gate)
    {
        SC_THREAD(do_nand);
        sensitive << a << b;
    }

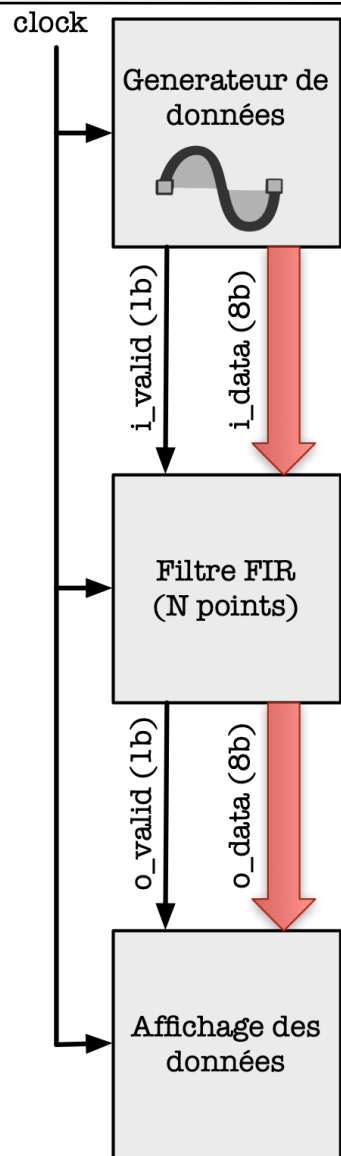
    void do_nand();
};
```



```
#include "NAND_Gate.h"

void NAND_Gate::do_nand(){
    while( true ){
        wait( );
        s = !(a && b);
    }
}
```

Exemple pédagogique (VHDL)

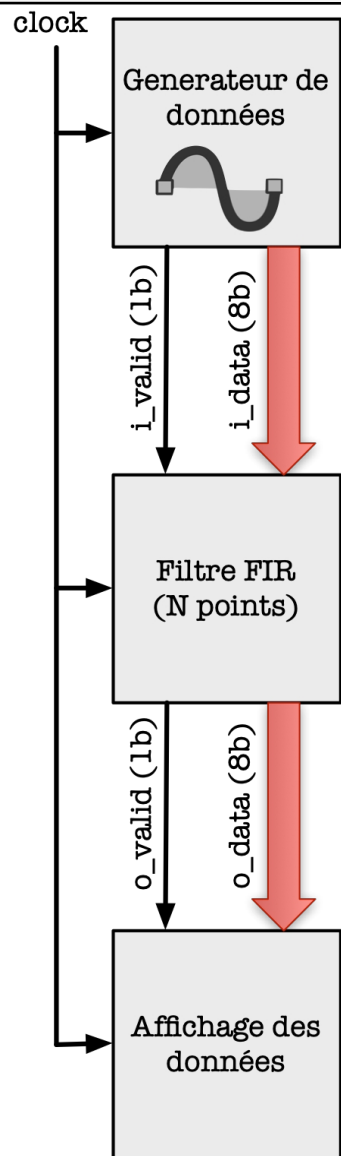


```
architecture Behavioral of Generator is
    --ROM for storing the sine values generated by MATLAB.
    type memory_type is array (0 to 29) of integer range -128 to 127;
    signal sine : memory_type :=(
        0, 16, 31, 45, 58, 67, 74, 77, 77, 74,
        67, 58, 45, 31, 16, 0, -16, -31, -45, -58,
        -67, -74, -77, -77, -74, -67, -58, -45, -31, -16);

    signal i : integer range 0 to 30:=0;
BEGIN

    PROCESS(CLOCK)
    BEGIN
        if(rising_edge(CLOCK)) then
            OUTPUT <= STD_LOGIC_VECTOR( TO_SIGNED(sine(i), 8) );
            i <= i+ 1;
            if(i = 29) then
                i <= 0;
            end if;
            o_VALID <= '1';
        END IF;
    END PROCESS;
END Behavioral;
```

Exemple pédagogique (VHDL comportemental)

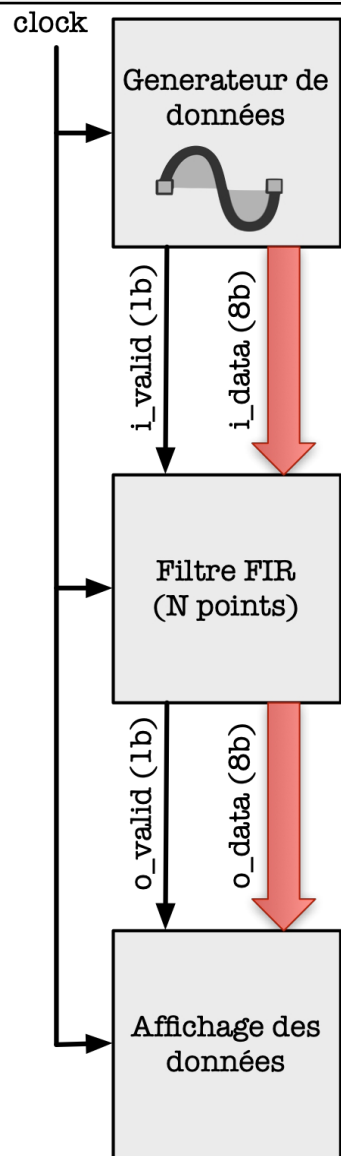


```
ARCHITECTURE Behavioral of Filtre is
    SIGNAL MEMORY : STD_LOGIC_VECTOR(7 downto 0);
BEGIN

    PROCESS (CLOCK)
        VARIABLE DATA_1 : SIGNED(7 downto 0);
        VARIABLE DATA_2 : SIGNED(7 downto 0);
        VARIABLE TEMP    : SIGNED(8 downto 0);
    BEGIN
        IF CLOCK'EVENT AND CLOCK = '1' THEN
            IF i_VALID = '1' THEN
                DATA_1 := SIGNED( INPUT );
                DATA_2 := SIGNED( MEMORY );
                TEMP    := RESIZE(DATA_1, 9) + RESIZE(DATA_2, 9);
                OUTPUT <= STD_LOGIC_VECTOR( TEMP(8 DOWNT0 1) );
                MEMORY <= INPUT;
                o_VALID<= '1';
            ELSE
                o_VALID<= '0';
            END IF;
        END IF;
    END PROCESS;

END Behavioral;
```

Exemple pédagogique (VHDL comportemental)

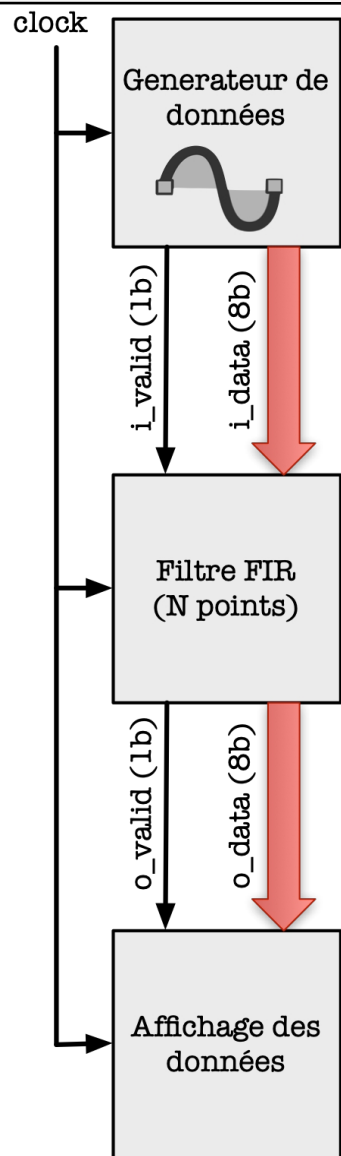


```
architecture Behavioral of Afficheur is
BEGIN

    PROCESS(CLOCK)
        VARIABLE value: INTEGER range -128 to 127;
    BEGIN
        IF CLOCK'EVENT AND CLOCK = '1' THEN
            IF i_VALID = '1' THEN
                value := TO_INTEGER( SIGNED( INPUT  ) );
                REPORT "SORTIE = " & INTEGER'IMAGE( value );
            END IF;
        END IF;
    END PROCESS;

END Behavioral;
```

Exemple pédagogique (VHDL comportemental)



```
architecture Behavioral of Afficheur is
BEGIN

    PROCESS(CLOCK)
        VARIABLE value: INTEGER range -128 to 127;
    BEGIN
        IF CLOCK'EVENT AND CLOCK = '1' THEN
            IF i_VALID = '1' THEN
                value := TO_INTEGER( SIGNED( INPUT  ) );
                REPORT "SORTIE = " & INTEGER'IMAGE( value );
            END IF;
        END IF;
    END PROCESS;

END Behavioral;
```

Exemple pédagogique (VHDL comportemental)

- ⊙ L'exemple sera repris en TP et écrit en SystemC sur plusieurs niveaux de modélisation

Conclusion sur l'exemple pédagogique

- ⊙ L'ensemble des étapes de raffinement ne sont pas obligatoires, elles dépendent essentiellement du système

Certaines étapes de raffinement peuvent être réalisées en même temps (types des données sur les bus + ordre de production et de consommation des données)

Une fois les modèles fonctionnels établis, une implantation particulière pour répondre aux besoins => description au cycle près

Certains composants vont imposer des contraintes sur leurs voisins (dynamique des données, type des interfaces, débit et ordre des transferts de données, etc.)

- ⊙ L'objectif n'est pas toujours d'aller vite mais de s'assurer qu'à chaque prise de décision, le système reste fonctionnel et répond aux contraintes

Les canaux de communication

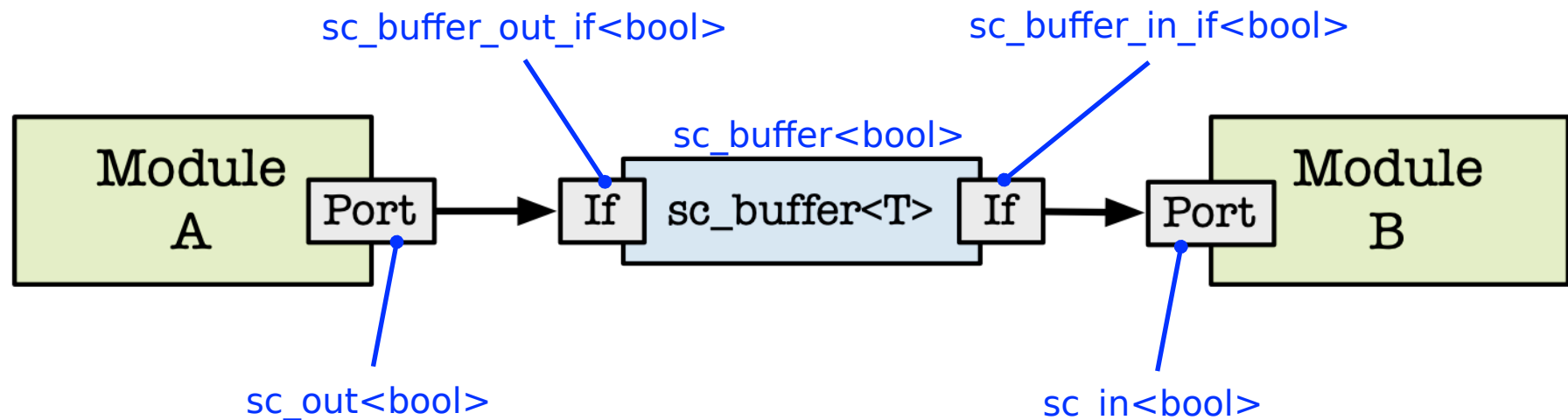
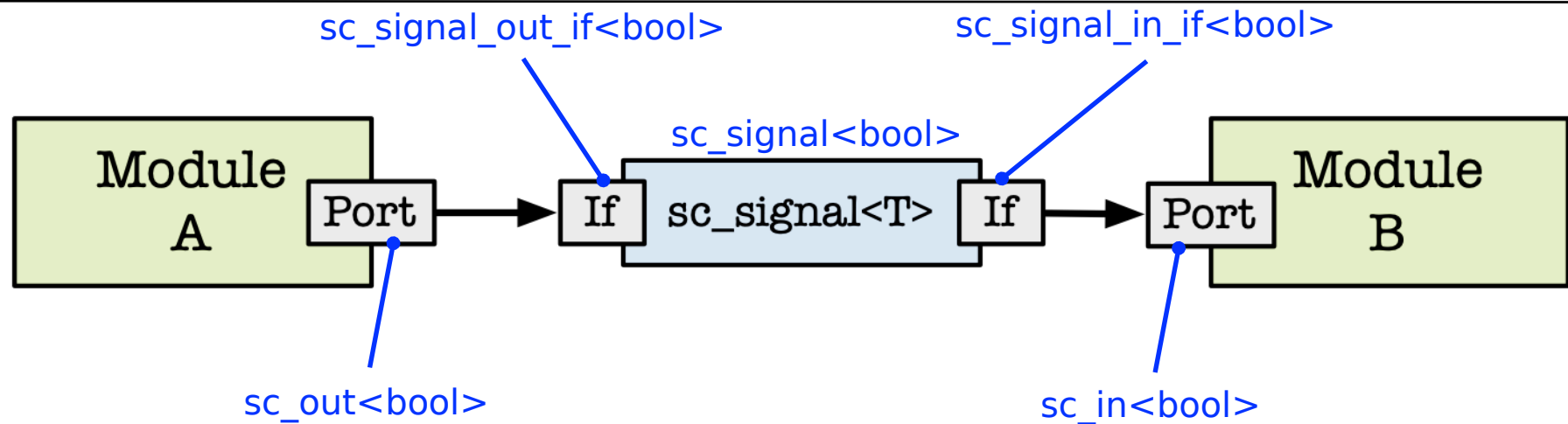
- ⊙ Les **canaux de communication** sont les moyens mis à disposition par SystemC afin de permettre le transfert de données entre différents modules composant le système.
- ⊙ On distingue de 2 liens de communication de niveau «physique»,
 - sc_signal<T>** : les signaux sont les canaux de communication les plus simples car ils modélisent de “simples fils”.
 - sc_buffer<T>** : ces canaux de communication sont similaires aux **sc_signal<T>** à une nuance prête: à chaque écriture (write), ils produisent un événement même si les données sont identiques (ce qui n'est pas le cas de **sc_signal<T>**).
- ⊙ Les canaux sont connectables uniquement aux ports qui sont compatibles aux interfaces de ces derniers.

⊙ Exemples

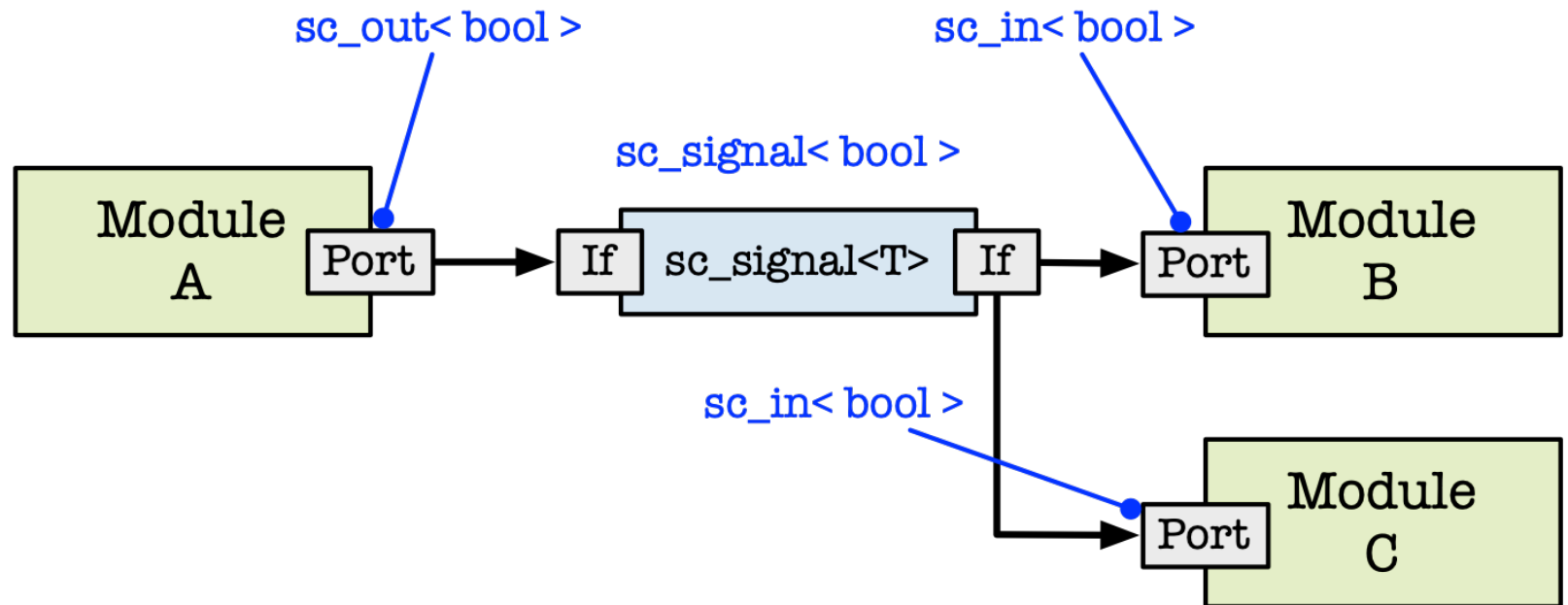
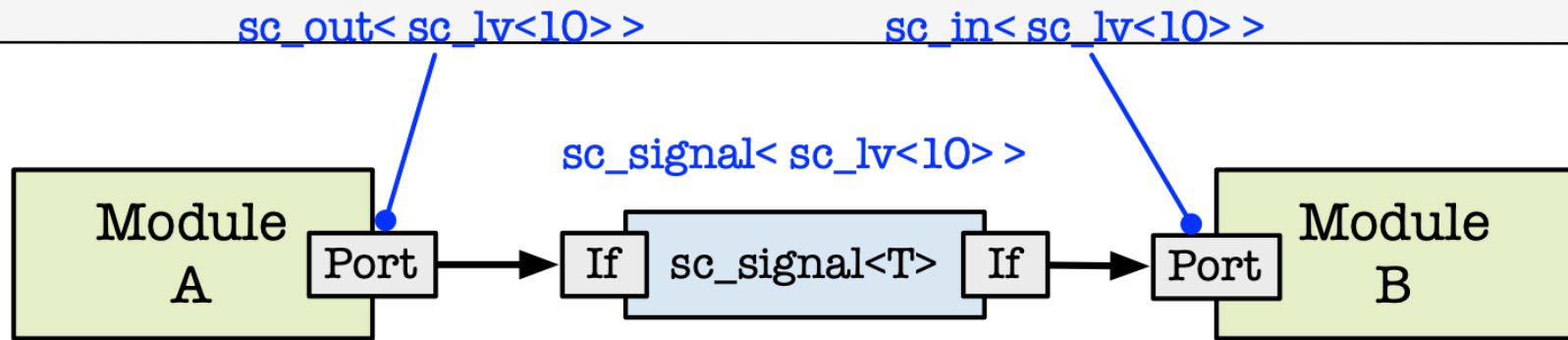
Un fil sur 1 bit (par exemple l'horloge de l'exemple ci-dessus) sera modélisé par un signal de type **sc_signal<bool>**.

Un bus de données ou d'adresse sur 32 bits sera défini comme étant un canal de type **sc_signal<sc_uint<32>>** ou **sc_signal<sc_lv<32>>**

Cohérence entre les ports d'E/S et canaux de communication



Exemples de liens entre modules



Modélisation (comportementale) d'une porte XOR

```
#include "systemc.h"

SC_MODULE(Porte_XOR_2)
{
    sc_in<bool> a, b;
    sc_out<bool> s;

    SC_CTOR(Porte_XOR_2)
    {
        SC_METHOD(do_xor);
        sensitive << a << b;
    }

    void do_xor();
};
```

On reste au niveau de
l'expression du comportement.

```
#include "Porte_XOR_2.h"

void Porte_XOR_2::do_xor(){
    s = (a && (!b)) || ((!a) && b);
}
```

Modélisation (structurelle) d'une porte XOR

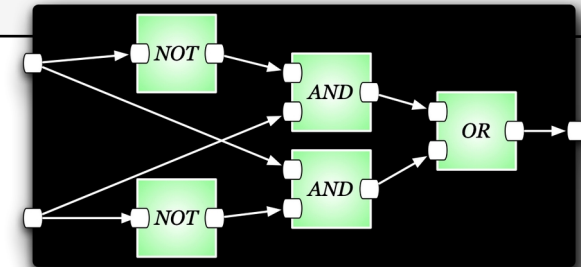
```
SC_MODULE(Porte_XOR)
{
    private:
        Porte_NOT *not_1;
        Porte_NOT *not_2;
        Porte_AND *and_1;
        Porte_AND *and_2;
        Porte_OR *or_1;
        sc_signal<bool> *signal_1;
        sc_signal<bool> *signal_2;
        sc_signal<bool> *signal_3;
        sc_signal<bool> *signal_4;
    public:
        sc_in<bool> a, b;
        sc_out<bool> s;

        SC_CTOR(Porte_XOR){
            // .....
            // .....
        }

        ~Porte_XOR(){
            // .....
            // .....
        }
};
```

Création des composants que l'on va interconnecter

```
not_1 = new Porte_NOT("NOT_1");
not_2 = new Porte_NOT("NOT_2");
and_1 = new Porte_AND("AND_1");
and_2 = new Porte_AND("AND_2");
or_1 = new Porte_OR ("OR_1");
signal_1 = new sc_signal<bool>();
signal_2 = new sc_signal<bool>();
signal_3 = new sc_signal<bool>();
signal_4 = new sc_signal<bool>();
not_1->a( a );
not_1->s( *signal_1 );
not_2->a( b );
not_2->s( *signal_2 );
and_1->a( a );
and_1->b( *signal_2 );
and_1->s( *signal_3 );
and_2->a( *signal_1 );
and_2->b( b );
and_2->s( *signal_4 );
or_1->a( *signal_3 );
or_1->b( *signal_4 );
or_1->s( s );
```

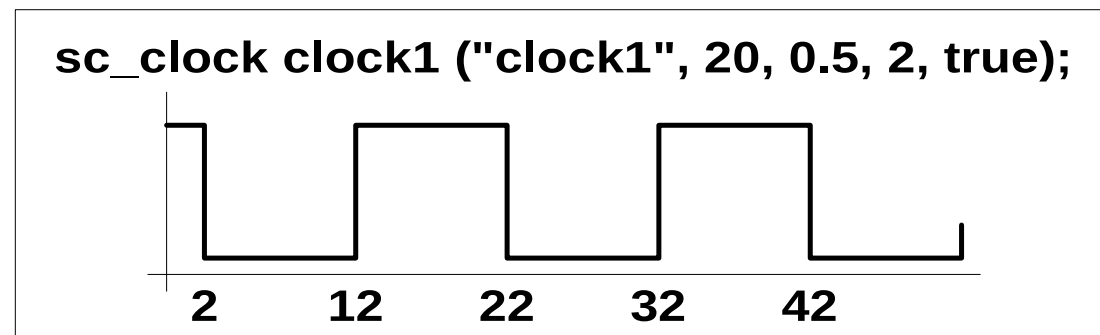


Destruction des composants

```
delete signal_1;
delete signal_2;
delete signal_3;
delete signal_4;
delete not_1;
delete not_2;
delete and_1;
delete and_2;
delete or_1;
```

Horloges et sensibilité

- `sc_clock clock_name ("clock_label", period, duty_ratio, offset, initial_value );`
- Sensitive positive, sensitive negative : au front montant et front descendant d'horloge
- Exemple:



Modélisation du temps en SystemC

- ⊙ Une classe nommée **sc_time** permet de modéliser le temps
 - Couple (valeur, unité temporelle)
- ⊙ Différentes résolutions (unités)

SC_SEC : seconde

SC_MS : milliseconde

SC_US : microseconde

SC_NS : nanoseconde

SC_PS : picoseconde

SC_FS : femtoseconde

```
sc_time periode(10, SC_NS);  
sc_time delta (1, SC_FS);  
sc_time latence;
```

```
latence = 10 SC_NS; // ERREUR !
```

La définition d'une valeur temporelle n'est possible que lors de l'instanciation d'un objet de type **sc_time**

Utilisation de la classe `sc_time`

- ⊙ SystemC a surchargé un certain nombre d'opérateurs de base du langage
- ⊙ La classe `sc_time` supporte ainsi les opérations suivantes :
 - Affectation
 - Addition
 - Soustraction
 - Multiplication
 - Comparaisons (égal, supérieur, etc.)
 - Affichage dans un flux standard

```
sc_time periode(10, SC_NS);
sc_time delta (1, SC_FS);

sc_time latence;

sc_time borne_min;
borne_min = periode - delta;

sc_time borne_max;
borne_max = periode + delta;

sc_time delay;
if( periode > latence ){
    delay = latence;
}else{
    delay = 2 * delta;
}
```

Modélisation des horloges à l'aide de la classe `sc_clock`

- ⊙ SystemC donne la possibilité de déclarer des horloges. C'est l'ordonnanceur qui se charge alors de la commutation des signaux. Le constructeur accepte les paramètres suivants:

Une chaîne de caractère précisant le nom de l'horloge,

La durée d'une période de l'horloge,

Au besoin le rapport cyclique de l'horloge s'il est différent de 50%.

```
// instancie une horloge de période 10ns
sc_clock clock1("clk100", 10, SC_NS);
sc_time t10(10, SC_NS);
sc_clock clock2("clk100", t10);

// instancie une horloge de période 15.3ns, de rapport cyclique 40%,
// démarrant au temps 45ms et dont le premier état est haut
sc_time tt(45, SC_MS);
sc_clock clock2("clk2", 15.3, SC_NS, 0.4, tt, true);
```

Utilisation du temps (sc_time) dans les processus

- ⊙ La modélisation du temps permet l'expression de délais d'exécution dans les processus

Etude des performances temporelles & de la synchronisation du système

- ⊙ Les spécifications temporelles sont utilisées afin d'exprimer :

Le temps passé à réaliser les calculs

Le temps d'attente avant d'entreprendre une nouvelle action

- ⊙ Pas de réelle notion de temps dans les SC_METHOD (!)

```
void do_gen(){  
    while( true ){  
        state = ! state;  
        out.write( state );  
        wait(5, SC_NS);  
    }  
}
```

Générateur d'horloge

```
void do_FeuRouge(){  
    while( true ){  
        out.write( FEU_VERT );  
        wait(30, SC_S);  
        out.write( FEU_ORANGE );  
        wait(5, SC_S);  
        out.write( FEU_ROUGE );  
        wait(60, SC_S);  
    }  
}
```

Gestion d'un feu rouge

Gestion des simulations

- ⊙ Afin de vérifier la fonctionnalité du système, il faut réaliser des simulations fonctionnelles
- ⊙ La bibliothèque SystemC inclut:

Un simulateur de type événementiel

Des commandes qui permettent d'interagir avec le simulateur (démarrer, régler la durée de simulation et suivre l'état d'avancement, etc.).

```
sc_start();                // Simulation infinie

sc_start(200, SC_NS);       // Simulation de 200ns

// Affiche le temps courant du simulateur
cout << "Actual time : " << sc_time_stamp() << endl;
```

Gestion des simulations (2)

- ⊙ The SystemC application has three phases/stages of operation:
- ⊙ 1. Elaboration: execution of statements prior to `sc_start()`.
 - ⊙ The primary purpose is to create internal data structures to support the semantics of simulation.
 - ⊙ During elaboration, the parts of the module hierarchy (modules, ports, primitive channels, and processes) are created, and ports and exports are bound to channels.
- ⊙ 2. Execution: further break-down to two stages:
 - ⊙ a) Initialization
 - ⊙ simulation kernel identifies all simulation processes and place them in either a runnable or waiting process set.
 - ⊙ All simulation processes are in runnable set except those requesting "no initialization".
 - ⊙

Gestion des simulations (3)

- ⊙
- ⊙ b) Simulation
- ⊙ is commonly described as a state machine that schedules processes to run, and advances simulation time. It has two internal phases:
 - ⊙ 1) evaluate: run all runnable processes one at a time. Each process runs till reaches wait() or return. Stops if no runnable processes left.
 - ⊙ 2) advance-time: once the set of runnable processes is emptied, simulation enters advance-time phase where it does:
 - ⊙ a) move simulated time to the closest time with a scheduled event
 - ⊙ b) move processes waiting for that particular time into the runnable set
 - ⊙ c) return to evaluation phase
 - ⊙

Gestion des simulations

The progression from evaluate to advance-time continues until one of the three things occurs. Then it moves to the cleanup phase.

- ⊙ a) all processes have yielded
- ⊙ b) a process has executed `sc_stop()`
- ⊙ c) maximum time is reached
- ⊙ 3. Cleanup or post-processing: destroy objects, releases memory, close open files etc.
- ⊙

Gestion des simulations

- ⊙ Four callback functions are called by the kernel at various stages during elaboration and simulation. They have the following declarations:
- ⊙ 1. virtual void before_end_of_elaboration(): called after the construction of the module hierarchy
- ⊙ 2. virtual void end_of_elaboration(): called at the very end of elaboration after all callbacks to before_end_of_elaboration have completed and after the completion of any instantiation or port binding performed by those callbacks and before starting simulation.
- ⊙ 3. virtual void start_of_simulation():
 - ⊙ a) called immediately when the application calls sc_start for the first time or at the very start of simulation, if simulation is initiated under the direct control of the kernel.
 - ⊙ b) if an application makes multiple calls to sc_start, start_of_simulation is called on the first call to sc_start.
 - ⊙ c) called after the callbacks to end_of_elaboration and before invoking the initialization phase of the scheduler.
- ⊙ 4. virtual void end_of_simulation():
 - ⊙ a) called when the scheduler halts because of sc_stop or at the very end of simulation if simulation is initiated under the direct control of the kernel.
 - ⊙ b) called only once even if sc_stop is called multiple times.

Exemple programme «main» lançant une simulation

```
int sc_main (int argc, char * argv []){  
    Gene g1("Data_Generator_1");  
    Gene g2("Data_Generator_2");  
    Adder add("Adder_1");  
    Terminal term("Terminal_1");  
  
    sc_signal<int> in1;  
    sc_signal<int> in2;  
    sc_signal<int> out;  
  
    g1.s(in1); g2.s(in2);  
    add.a(in1); add.b(in2); add.s(out);  
    term.a(out);  
  
    sc_start(200, SC_NS);  
  
    return 1;  
}
```

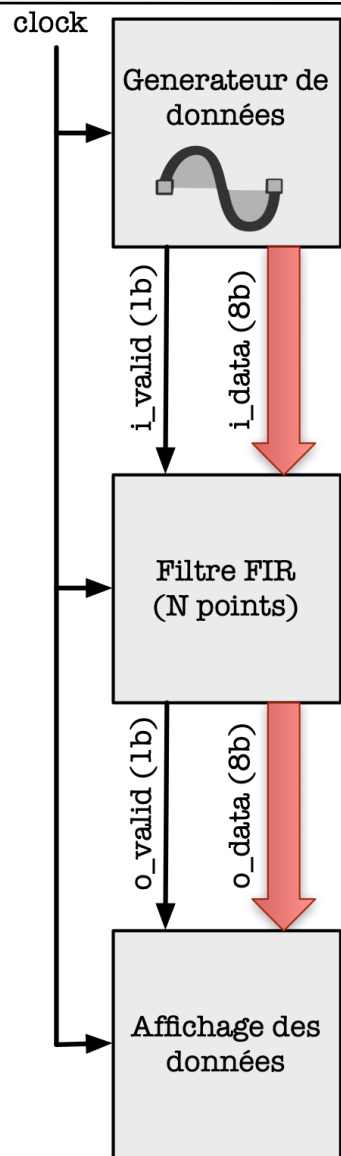
Déclaration des composants qui vont être simulés

Déclaration des canaux de communication

Liaison des ports des modules aux canaux de communication

Lancement d'une simulation sur une durée de 200ns

Exemple: écriture du «programme» de simulation (sc_main)



```
#include "../modules/top_module.h"
#include <iostream>

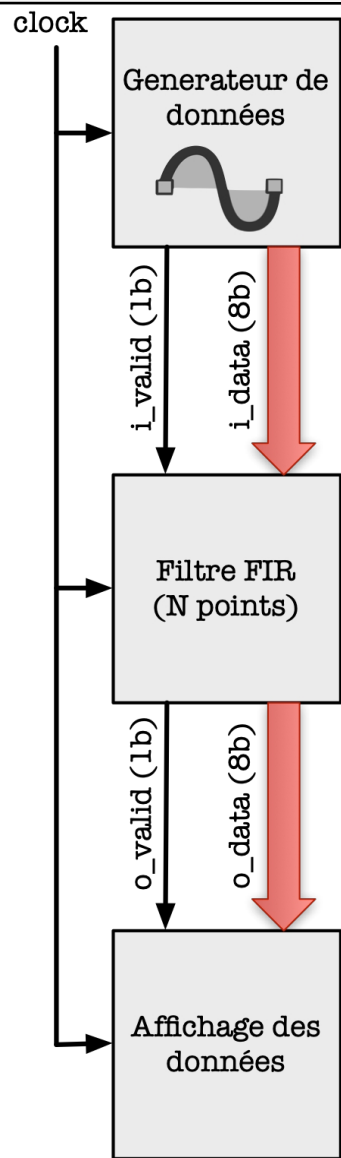
using namespace std;

int main (int argc, char * argv []){
    top_module top("top");

    sc_start(400,SC_NS);

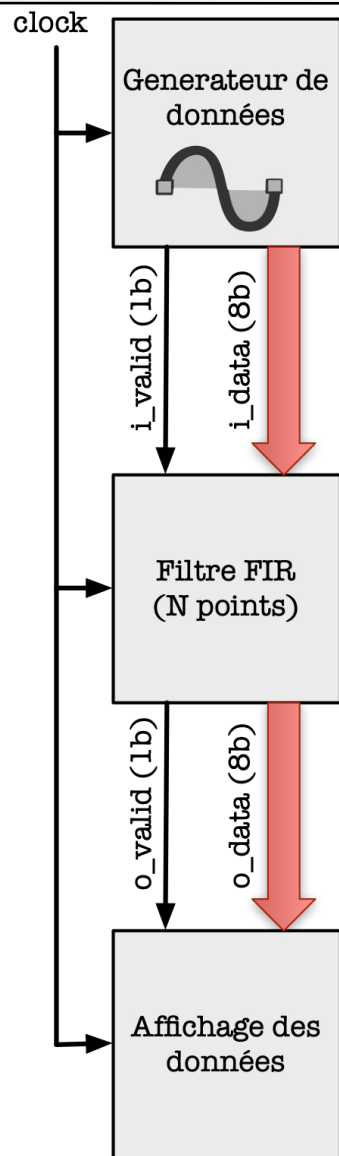
    return 0;
}
```

Résultats observés post-simulation (terminal)



```
./bin/main
WARNING: Default time step is used for VCD tracing.
Time = 20 ns / out = 0
Time = 40 ns / out = 8
Time = 60 ns / out = 23
Time = 80 ns / out = 38
Time = 100 ns / out = 51
Time = 120 ns / out = 62
Time = 140 ns / out = 70
Time = 160 ns / out = 75
Time = 180 ns / out = 77
Time = 200 ns / out = 75
Time = 220 ns / out = 70
Time = 240 ns / out = 62
Time = 260 ns / out = 51
Time = 280 ns / out = 38
```

Résultats observés post-simulation (Xilinx ISim)



ISim P.40xd (signature 0x8ef4fb42)

This is a Full version of ISim.

Time resolution is 1 ps

Simulator is doing circuit initialization process.

Finished circuit initialization process.

at 25 ns(1), Instance /tb_system/uu3/ : Warning: NUMERIC_STD.TO_INTEGER

at 25 ns(1): Note: SORTIE = 0 (/tb_system/uu3/).

at 35 ns(1): Note: SORTIE = 8 (/tb_system/uu3/).

at 45 ns(1): Note: SORTIE = 23 (/tb_system/uu3/).

at 55 ns(1): Note: SORTIE = 38 (/tb_system/uu3/).

at 65 ns(1): Note: SORTIE = 51 (/tb_system/uu3/).

at 75 ns(1): Note: SORTIE = 62 (/tb_system/uu3/).

at 85 ns(1): Note: SORTIE = 70 (/tb_system/uu3/).

Observation des signaux internes au modèle

- ⊙ Il peut être nécessaire de conserver une traces de l'évolution temporelle des signaux dans les composants pour deboguer ou conserver une trace du fonctionnement du système

Il existe un ensemble de routines permettant de générer des chronogrammes
(fichiers au format VCD),

- ⊙ Pour générer un chronogramme (vcd), il faut :

Créer le fichier qui va contenir les données

```
sc_trace_file * ma_trace = sc_create_vcd_trace_file( "trace.vcd" );
```

Ajouter les signaux que l'on souhaite pouvoir analyser par la suite,

```
sc_trace(ma_trace, clk, "signal_clock");
```

Lancer la simulation du système,

```
sc_start(200, SC_NS);
```

Fermer proprement le fichier contenant les signaux avant de quitter,

```
sc_close_vcd_trace_file( ma_trace );
```

Exemple de mémorisation des signaux

```
sc_clock clk("TestClock", 10, SC_NS, 0.5);
sc_signal<bool> d1_d2;
sc_signal<bool> d2_d3;
sc_signal<bool> d3_d4;
sc_signal<bool> sortie;
```

```
DivFreq *d1 = new DivFreq("Diviseur_1");
DivFreq *d2 = new DivFreq("Diviseur_2");
DivFreq *d3 = new DivFreq("Diviseur_3");
DivFreq *d4 = new DivFreq("Diviseur_4");
```

```
// Interconnexion des composants ....
```

```
sc_trace_file *trace = sc_create_vcd_trace_file("My_wave_form");
sc_trace(trace, clk, "Horloge");
sc_trace(trace, d1_d2, "Division_1");
sc_trace(trace, d2_d3, "Division_2");
sc_trace(trace, d3_d4, "Division_3");
sc_trace(trace, sortie, "Division_4");
```

```
sc_start(200, SC_NS);
```

```
sc_close_vcd_trace_file( trace );
```

```
return 0;
```

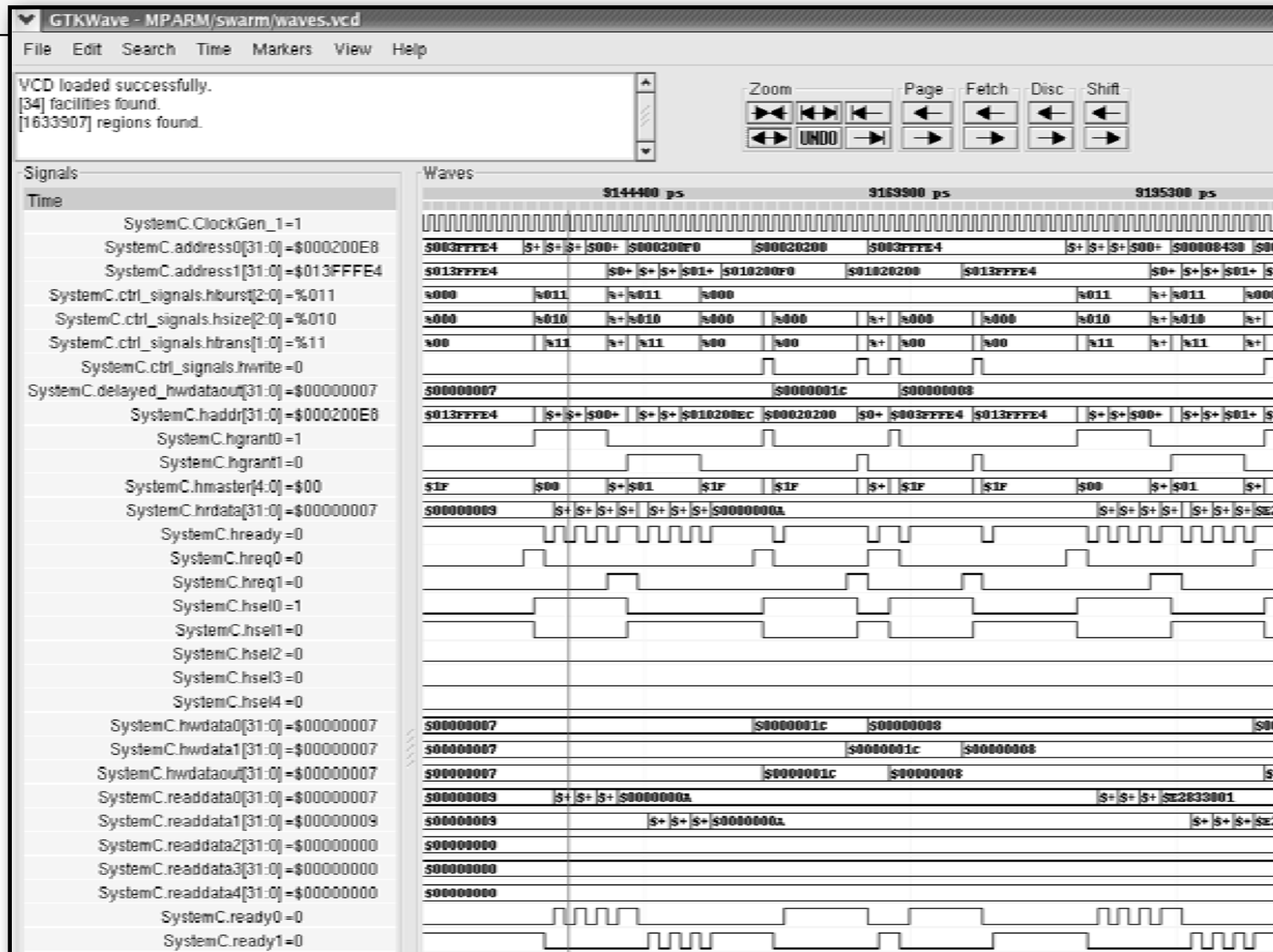
Création du fichier

Enregistrement de
signaux à observer

Lancement de la
simulation

Fermeture du
fichier

Exemple d'affichage des signaux avec Gtkwave



Impacte des différentes approches sur le simulateur

- ⊙ La différence majeure qui existe entre ces 3 modèles de processus que nous venons de décrire réside dans les conditions de leur déclenchement

SC_METHOD : liste de sensibilité statique sur les entrées du module

SC_THREAD : liste de sensibilité dynamique sur les entrées + délais

(SC_CTHREAD : actif uniquement sur les fronts d'une horloge définie statiquement)

- ⊙ Entre chaque phase de calcul, le simulateur doit déterminer quels sont les processus à réveiller afin d'effectuer le traitement attendu

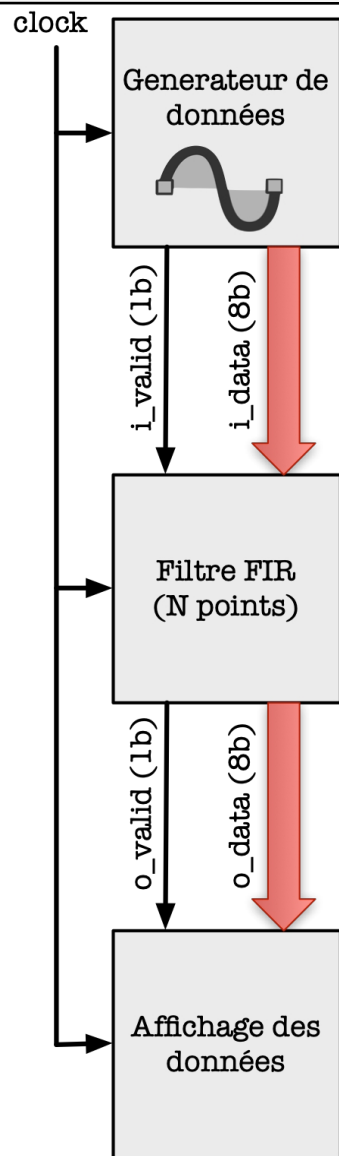
Cette étape est consommatrice de temps CPU dans les systèmes complexes

Réveiller un processus et l'exécuter si rien ne s'est produit est inefficace !

- ⊙ Le choix du type de processus n'est pas anodin car il va impacter fortement sur les performances de simulation

La majorité du temps de simulation peut être utilisée uniquement par le noyau du simulateur SystemC afin de réveiller, endormir des processus

Exemple: écriture du «programme» de simulation (sc_main)



```
#include "../modules/top_module.h"
#include <iostream>

using namespace std;

int main (int argc, char * argv []){
    top_module top("top");

    sc_trace_file *trace = sc_create_vcd_trace_file( "trace" );

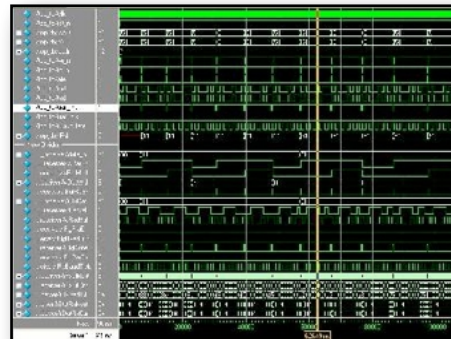
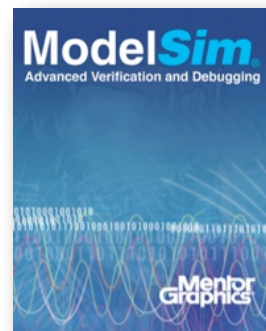
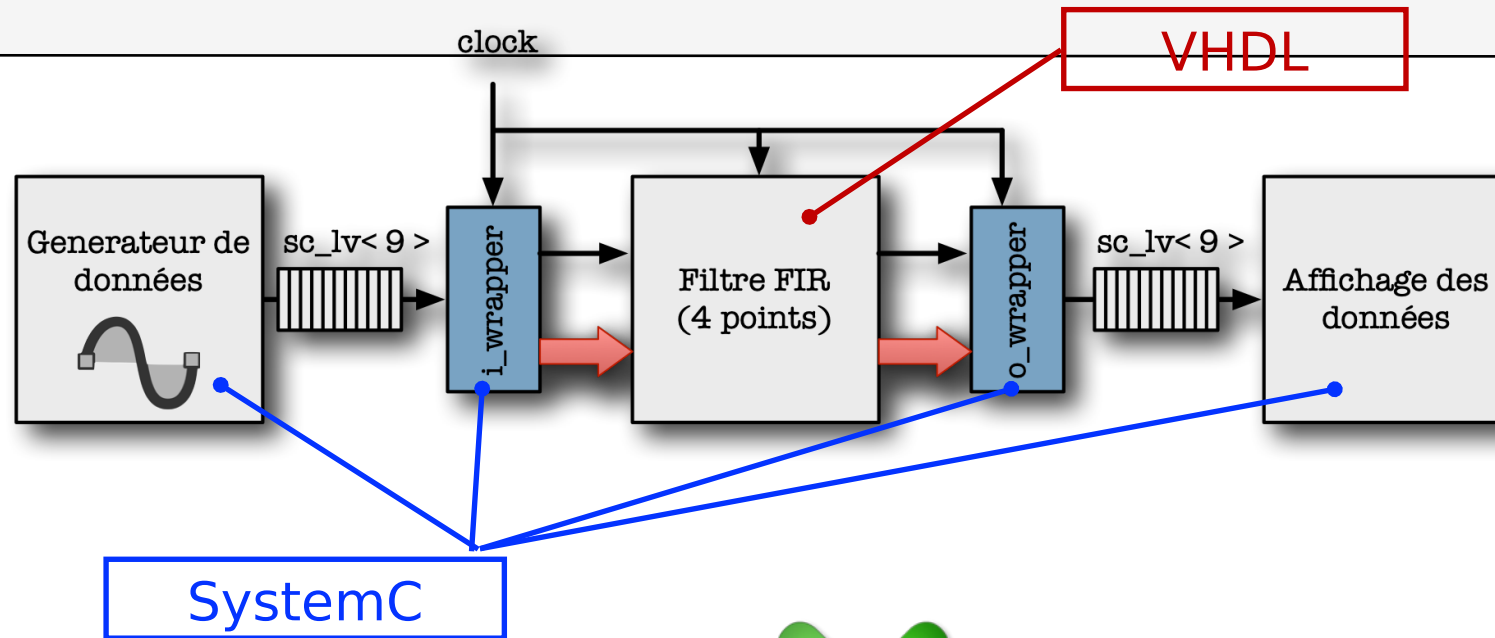
    sc_trace(trace, *(top.clk), "clock");
    sc_trace(trace, *(top.s1), "input");
    sc_trace(trace, *(top.s2), "outpt");
    sc_trace(trace, *(top.v1), "invalid");
    sc_trace(trace, *(top.v2), "ovalid");

    sc_start(400, SC_NS);

    sc_close_vcd_trace_file( trace );

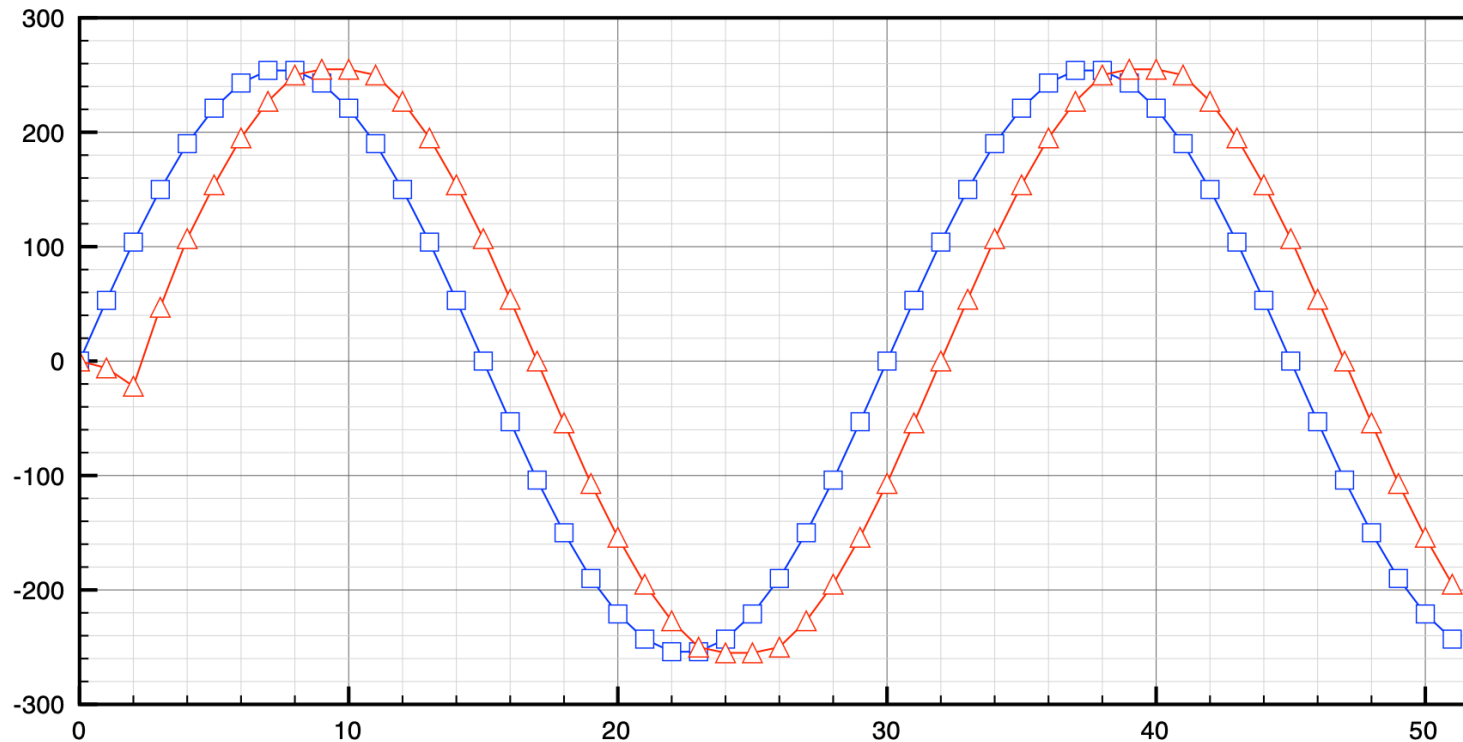
    return 0;
}
```

La simulation conjointe SystemC <=> VHDL



Vérification
fonctionnelle de
l'architecture

Validation du comportement post-simulation conjointe



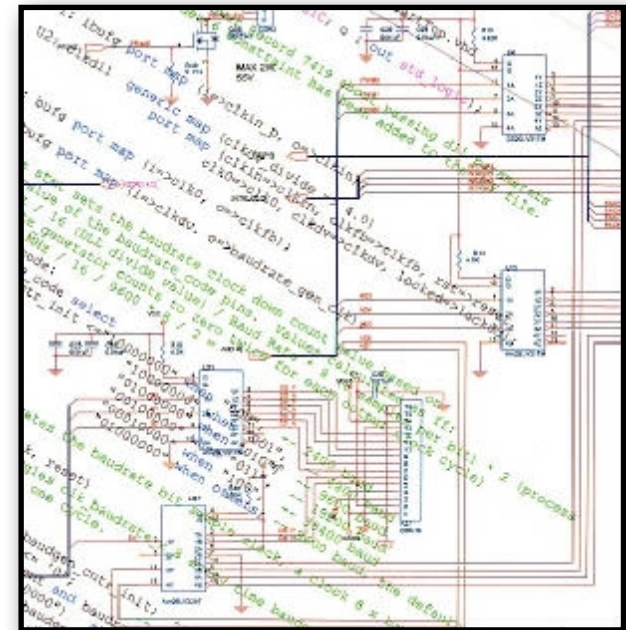
Un seul et unique modèle

- Mêmes entrées, et donc mêmes sorties
- Gain en temps & en fiabilité

Conclusion intermédiaire



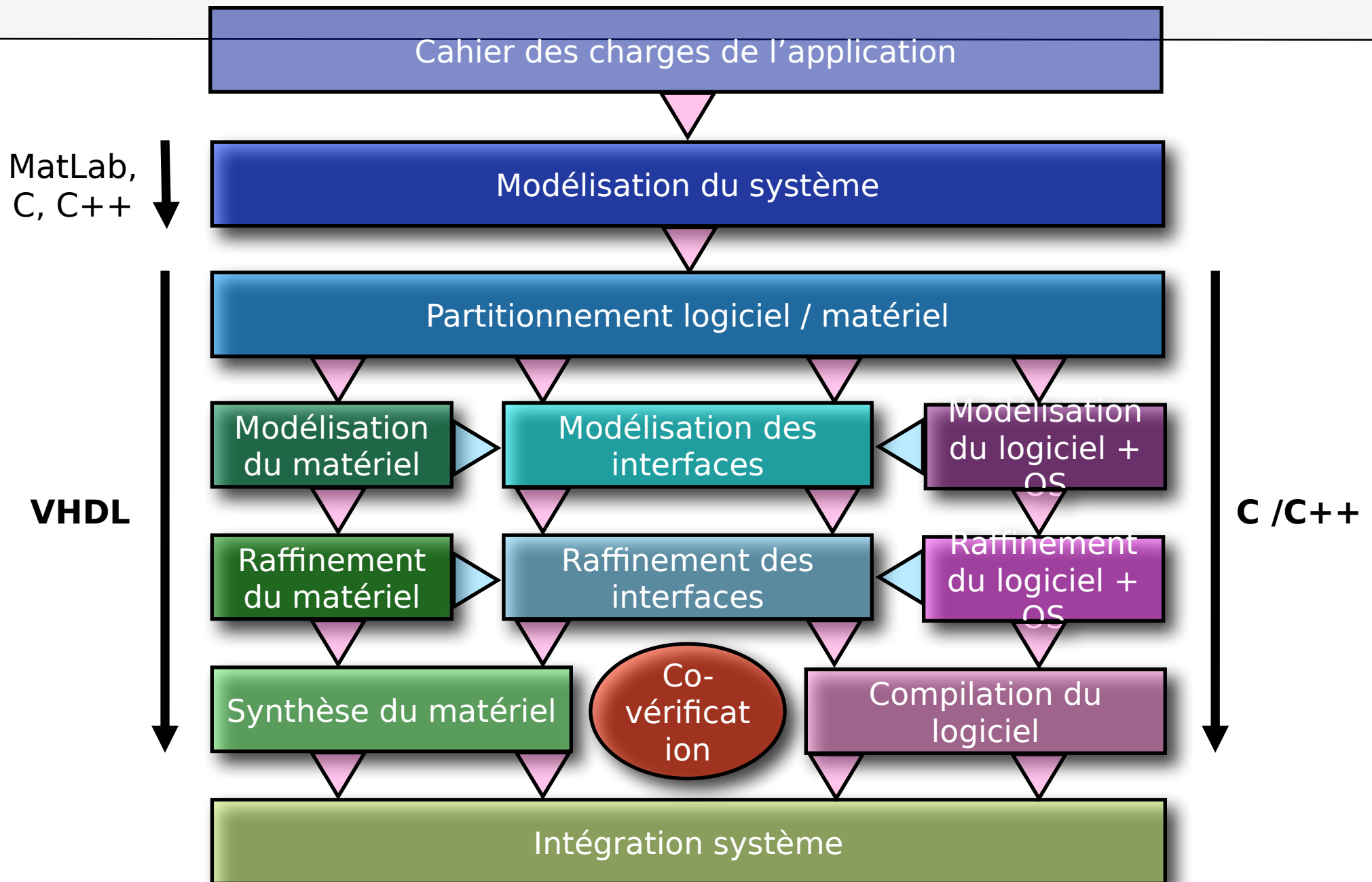
VHDL



Conclusion sur le langage SystemC

- ⊙ Le langage *SystemC* offre aux concepteurs de systèmes une grande liberté dans la modélisation des systèmes à concevoir
- ⊙ Le flot de raffinement progressif à l'aide d'un seul et unique langage (jusqu'à l'implantation) permet de gagner en temps de développement
 - Moins d'erreurs de codage dans conversions (C => VHDL),
 - Les étapes sont plus simples => moins d'erreurs de raffinement
- ⊙ Il existe d'autres langages concurrents à SystemC comme SystemVerilog qui est défendu par les USA (SystemC est européen)
- ⊙ Il existe encore des lacunes dans le flots de conception : aide au raffinement, éditeurs performants, débogueurs confortables, outils d'analyse et de synthèse logique, etc.

Flot de conception système usuel



Evolution du flot de conception à l'aide de SystemC

